

Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores
(Computer Engineering School)

Programa de Licenciatura en Ingeniería en Computadores
(Licentiate Degree Program in Computer Engineering)



**Diseño e Integración de un Módulo PLPUF Programable para
Sistemas Embebidos**
(Design and Integration of a Programmable PLPUF Module for Embedded Systems)

**Informe de Trabajo de Graduación para optar por el título de
Ingeniero en Computadores con grado académico de
Licenciatura**

(Report of Graduation Work in fulfillment of the requirements for the degree of Licentiate in Computer Engineering)

Emanuel Antonio Marín Gutiérrez

Cartago, Junio, 2026
(Cartago, June, 2026)

Dedicatoria

A mi querida mamá, a mi familia y a mis amigos.

Agradecimientos

A mi querida mamá, a mi familia y a mis amigos, por su apoyo incondicional a lo largo de toda mi formación universitaria. Su confianza y paciencia fueron el motor que me permitió llegar hasta aquí.

A los profesores de la Escuela de Ingeniería en Computadores y la Escuela de Ingeniería en Electrónica del Instituto Tecnológico de Costa Rica, quienes con su dedicación y exigencia me transmitieron no solo conocimientos técnicos, sino también la disciplina y la curiosidad necesarias para enfrentar problemas complejos. Cada curso, cada laboratorio y cada proyecto contribuyó a forjar al ingeniero que soy hoy.

Agradezco especialmente al Dr. Jeferson González Gómez, quien me asignó este trabajo final de graduación y confió en mis capacidades para llevarlo a cabo. Su conocimiento en el área de seguridad de hardware fue determinante para definir el rumbo del proyecto.

A todos quienes de una u otra forma contribuyeron a la realización de este proyecto, mi más sincero agradecimiento.

Emanuel Antonio Marín Gutiérrez
Cartago, June, 2026

Resumen

Este proyecto presenta el diseño, implementación e integración de un módulo de Función Físicamente No Clonable basado en la arquitectura Pseudo-LFSR (PLPUF) como periférico dentro de un *System-on-Chip* (SoC) sobre FPGA. El módulo explota las variaciones físicas inherentes al proceso de fabricación del silicio para generar identificadores de hardware únicos de 128 bits, eliminando la necesidad de almacenar claves criptográficas en memorias no volátiles. El sistema completo integra el PLPUF como periférico AXI4-Lite controlado por el procesador suave MicroBlaze V sobre la plataforma Basys3 con FPGA Artix-7, permitiendo la evaluación automatizada de pares desafío-respuesta mediante comunicación UART.

Para el desarrollo del proyecto se utilizó el lenguaje de descripción de hardware Verilog para la implementación del núcleo PLPUF y la interfaz AXI4-Lite, AMD Vivado para la síntesis, implementación y generación del *bitstream*, y AMD Vitis para el desarrollo del *firmware* de control en lenguaje C. La evaluación estadística del módulo se realizó mediante 10 ejecuciones independientes sobre un mismo dispositivo FPGA, acumulando más de 140 000 evaluaciones del PLPUF. Se midieron las métricas estándar de la literatura: uniformidad (47–51 %), confiabilidad (98.97 % para duración de activación $c = 1$), distancia de Hamming intra-dispositivo (1.31 bits) y distancia de Hamming inter-desafío (48.48 %).

En este documento se detalla el proceso de diseño, implementación, integración y evaluación del sistema, incluyendo los fundamentos teóricos de las PUFs y las métricas de seguridad empleadas, la metodología de desarrollo, los resultados experimentales obtenidos y su comparación con la literatura de referencia. Los resultados demuestran que el módulo implementado exhibe propiedades estadísticas consistentes con las reportadas para la arquitectura PLPUF original, validando su correcta portabilidad a la familia FPGA Artix-7 y aportando una plataforma abierta y reproducible para la investigación en seguridad de hardware basada en PUFs.

Palabras clave: FPGA, Artix-7, PUF, PLPUF, AXI4-Lite, MicroBlaze V, seguridad de hardware, *System-on-Chip*, Verilog, sistemas embebidos.

Abstract

This project presents the design, implementation and integration of a Physical Unclonable Function module based on the Pseudo-LFSR (PLPUF) architecture as a peripheral within an FPGA-based System-on-Chip (SoC). The module exploits the inherent physical variations of the silicon manufacturing process to generate unique 128-bit hardware identifiers, eliminating the need to store cryptographic keys in non-volatile memories. The complete system integrates the PLPUF as an AXI4-Lite peripheral controlled by the MicroBlaze V soft processor on the Basys3 platform with an Artix-7 FPGA, enabling automated challenge-response pair evaluation through UART communication.

For the development of this project, the Verilog hardware description language was used for the implementation of the PLPUF core and the AXI4-Lite interface, AMD Vivado for synthesis, implementation and bitstream generation, and AMD Vitis for the development of the control firmware in C. The statistical evaluation of the module was performed through 10 independent runs on a single FPGA device, accumulating over 140 000 PLPUF evaluations. Standard metrics from the literature were measured: uniformity (47–51 %), reliability (98.97 % for activation duration $c = 1$), intra-device Hamming distance (1.31 bits) and inter-challenge Hamming distance (48.48 %).

This document details the design, implementation, integration and evaluation process of the system, including the theoretical foundations of PUFs and the security metrics employed, the development methodology, the experimental results obtained and their comparison with the reference literature. The results demonstrate that the implemented module exhibits statistical properties consistent with those reported for the original PLPUF architecture, validating its correct portability to the Artix-7 FPGA family and providing an open and reproducible platform for hardware security research based on PUFs.

Keywords: FPGA, Artix-7, PUF, PLPUF, AXI4-Lite, MicroBlaze V, hardware security, System-on-Chip, Verilog, embedded systems.

Índice general

	Página
Índice de figuras	iv
Índice de tablas	vi
Lista de abreviaciones	vii
1 Introducción	1
1.1 Antecedentes del proyecto.....	2
1.1.1 Descripción de la institución.....	2
1.1.2 Área de conocimiento del proyecto	2
1.1.3 Trabajos similares	3
1.2 Planteamiento del problema	4
1.2.1 Contexto del problema	4
1.2.2 Justificación del problema	4
1.2.3 Enunciado del problema.....	5
1.3 Objetivos del proyecto.....	5
1.3.1 Objetivo general	5
1.3.2 Objetivos específicos.....	5
1.4 Alcances, entregables y limitaciones	6
1.4.1 Alcances	6
1.4.2 Entregables.....	6
1.4.3 Limitaciones.....	6
2 Marco de referencia teórico	7
2.1 Sistemas en chip (SoC)	7
2.1.1 Arquitectura MicroBlaze V	7
2.1.2 Bus AXI4-Lite	9
2.2 FPGAs y lógica reconfigurable.....	10
2.2.1 Familia Artix-7 y plataforma Basys 3	11
2.2.2 Herramientas: Vivado y Vitis.....	12
2.3 Funciones físicamente no clonables (PUF)	13
2.3.1 Fundamentos y taxonomía	13
2.3.2 PUFs programables: Pseudo-LFSR PUF (PLPUF).....	15
2.3.3 Modelos de desafío-respuesta (CRP)	18
2.4 Seguridad en sistemas embebidos.....	19

2.4.1	Autenticación de hardware.....	19
2.4.2	Métricas de evaluación: unicidad, confiabilidad y uniformidad	20
3	Marco metodológico.....	23
3.1	Estrategia metodológica.....	23
3.2	Diagrama de la secuencia del proceso	24
3.3	Propuesta metodológica por objetivo	25
3.4	Herramientas de desarrollo.....	25
3.4.1	Hardware: placa Basys3	25
3.4.2	AMD Vivado Design Suite.....	25
3.4.3	AMD Vitis	27
3.4.4	Terminal serial y Python.....	27
3.5	Diagrama general de la solución	27
3.6	Diseño del módulo PLPUF en HDL	29
3.6.1	Visión general del módulo.....	29
3.6.2	Cadena de inversores y retroalimentación.....	30
3.6.3	Máquina de estados finitos.....	31
3.6.4	Restricciones de síntesis	31
3.7	Diseño de la interfaz AXI4-Lite.....	31
3.7.1	Mapa de registros.....	32
3.7.2	Lógica del esclavo AXI	33
3.7.3	Empaquetado del IP.....	33
3.8	Integración del sistema SoC.....	34
3.8.1	Diseño de bloques en Vivado	34
3.8.2	Flujo de síntesis e implementación	35
3.8.3	Exportación a Vitis.....	36
3.9	Desarrollo del software de control	37
3.9.1	Driver en C para el PLPUF	37
3.9.2	Aplicación de validación básica (<code>test_app1</code>)	38
3.9.3	Aplicación de evaluación exhaustiva (<code>test_app2</code>)	39
3.10	Protocolo de evaluación del PLPUF.....	40
3.10.1	Recolección de pares desafío-respuesta	40
3.10.2	Cálculo de métricas.....	40
3.10.3	Efecto avalancha	41
4	Descripción del trabajo realizado	42
4.1	Proceso para llegar a la solución aplicada.....	42
4.1.1	OE1: Implementación del módulo PLPUF en HDL.....	42
4.1.2	OE2: Desarrollo de la interfaz AXI-slave	43
4.1.3	OE3: Integración del sistema SoC.....	43
4.1.4	OE4: Protocolo de evaluación estadística.....	44
4.2	Análisis de los resultados finales	44
4.2.1	Resultados de la implementación HDL (OE1).....	44

4.2.2	Pruebas de la interfaz AXI (OE2)	45
4.2.3	Prueba de integración del SoC (OE3)	46
4.2.4	Resultados de la evaluación de seguridad (OE4)	46
4.2.5	Comparación con referencias y discusión general.....	51
5	Conclusiones y recomendaciones	53
5.1	Conclusiones	53
5.2	Recomendaciones	55
A	Implicaciones sociales y éticas del proyecto	56
	Bibliografía.....	59

Índice de figuras

Página

2.1	Diagrama de bloques del procesador MicroBlaze V [1].	9
2.2	Canales de escritura del protocolo AXI: canal de dirección (AW), canal de datos (W) y canal de respuesta (B) [2].	10
2.3	Canales de lectura del protocolo AXI: canal de dirección (AR) y canal de datos (R) [2].	10
2.4	Placa de desarrollo Basys 3 con FPGA Artix-7 [3].	12
2.5	Estructura del Pseudo-LFSR PUF de 128 bits: cadena de celdas de lógica central con compuertas XOR en las posiciones de <i>tap</i> y retroalimentación desde <i>Dout</i> [1] hacia la primera celda [4].	16
2.6	Estructura interna de una celda de lógica central: multiplexor controlado por <i>SEL</i> e inversor como fuente de entropía [4].	16
2.7	Máquina de estados del PLPUF: ciclo de evaluación desafío- respuesta con las señales de control y las condiciones de tran- sición entre los estados <i>IDLE</i> , <i>LOAD</i> , <i>ACTIVE</i> y <i>DONE</i>	17
3.1	Metodología secuencial por fases del proyecto. Cada fase se alineja con un objetivo específico (OE) y produce entregables verificables que habilitan la fase siguiente.	24
3.2	Diagrama general de la solución: sistema SoC con PLPUF in- tegrado. Los colores agrupan los componentes por función y los números indican el flujo de operación.	28
3.3	Diagrama de primer nivel del módulo <i>plpuf_core</i> : interfaz de puertos con anchos de señal.	29
3.4	Diagrama del diseño de la interfaz AXI4-Lite: los canales AXI comunican el bus con el bloque de registros, que a su vez controla el núcleo PLPUF.	32
3.5	Jerarquía de módulos del IP AXI PLPUF: tres niveles de en- capsulamiento con el <i>wrapper</i> , el esclavo AXI4-Lite (registros y FSM AXI) y el núcleo PLPUF.	34

3.6	Diseño de bloques del sistema SoC en Vivado IP Integrator, mostrando la interconexión entre el procesador MicroBlaze V, la memoria local (BRAM/LMB), el módulo de depuración (MDM V), el AXI SmartConnect, el periférico UART Lite y el periférico PLPUF desarrollado en este proyecto.	36
3.7	Arquitectura en capas del software de control: las aplicaciones de prueba utilizan el <i>driver</i> del PLPUF, que a su vez accede a los registros del periférico mediante operaciones de lectura/escritura AXI. ...	37
3.8	Diagrama de flujo de la función <code>Plpuf_Evaluate()</code> : ciclo completo de evaluación desafío-respuesta.	38
4.1	Resumen de métricas del PLPUF en función de la duración de activación c . Las barras de error representan $\pm 1\sigma$ sobre 10 corridas independientes. (a) Uniformidad, (b) confiabilidad, (c) HD intra-dispositivo, (d) HD inter-desafío.	47
4.2	Uniformidad del PLPUF en función de la duración de activación c (media $\pm 1\sigma$, $n = 10$). La línea discontinua roja indica el valor ideal de 50 %.	48
4.3	Confiabilidad (eje izquierdo) y BER (eje derecho) en función de la duración de activación c (media $\pm 1\sigma$, $n = 10$).	49
4.4	Distancia de Hamming intra-dispositivo promedio en función de la duración de activación c (media $\pm 1\sigma$, $n = 10$).	49
4.5	Distancia de Hamming inter-desafío (como porcentaje de los 128 bits) en función de la duración de activación c (media $\pm 1\sigma$, $n = 10$). La línea discontinua roja indica el ideal de 50 %.	50
4.6	Efecto avalancha del PLPUF a $c = 5$: porcentaje de bits que cambian en la respuesta al invertir un solo bit del desafío (media $\pm 1\sigma$, $n = 10$). La línea discontinua roja indica el ideal de 50 % y la línea punteada verde el promedio observado.	51

Índice de tablas

Página

3.1	Propuesta metodológica por objetivo específico.	26
3.2	Interfaz de puertos del módulo <code>plpuf_core</code>	29
3.3	Parámetros configurables del módulo <code>plpuf_core</code>	30
3.4	Mapa de registros del periférico AXI PLPUF.	32
3.5	Mapa de direcciones del sistema SoC.	35
4.1	Utilización de recursos del sistema SoC completo sobre la FPGA Artix-7 XC7A35T tras la implementación física.	45
4.2	Utilización de recursos del periférico AXI PLPUF y del proce- sador MicroBlaze V, según el reporte de síntesis individual.	45
4.3	Resultados del barrido de duración (media $\pm$ desviación estándar, $n = 10$ corridas, 20 desafíos $\times$ 50 repeticiones por corrida).	47
4.4	Comparación de resultados con la evaluación de referencia de Hori et al. [4].	51

Lista de abreviaciones

AMBA	<i>Advanced Microcontroller Bus Architecture</i> – Arquitectura de bus de microcontrolador avanzado
ASIC	<i>Application-Specific Integrated Circuit</i> – Circuito integrado de aplicación específica
AXI	<i>Advanced eXtensible Interface</i> – Interfaz extensible avanzada
BER	<i>Bit Error Rate</i> – Tasa de error de bit
BRAM	<i>Block RAM</i> – Bloque de memoria RAM dentro de la FPGA
BSP	<i>Board Support Package</i> – Paquete de soporte de tarjeta
CRP	<i>Challenge-Response Pair</i> – Par desafío-respuesta
DRC	<i>Design Rule Check</i> – Verificación de reglas de diseño
DSP	<i>Digital Signal Processor</i> – Procesador digital de señales
FAR	<i>False Acceptance Rate</i> – Tasa de falsa aceptación
FF	<i>Flip-Flop</i> – Biestable
FPGA	<i>Field Programmable Gate Array</i> – Arreglo de compuertas programable en campo
FRR	<i>False Rejection Rate</i> – Tasa de falso rechazo
FSM	<i>Finite State Machine</i> – Máquina de estados finitos
HD	<i>Hamming Distance</i> – Distancia de Hamming
HDA	<i>Helper Data Algorithm</i> – Algoritmo de datos auxiliares
HDL	<i>Hardware Description Language</i> – Lenguaje de descripción de hardware
IoT	<i>Internet of Things</i> – Internet de las cosas
IP	<i>Intellectual Property</i> – Propiedad intelectual (núcleo IP)
ISA	<i>Instruction Set Architecture</i> – Arquitectura del conjunto de instrucciones
JTAG	<i>Joint Test Action Group</i> – Estándar IEEE 1149.1 de depuración
LFSR	<i>Linear Feedback Shift Register</i> – Registro de desplazamiento con retroalimentación lineal
LMB	<i>Local Memory Bus</i> – Bus local de memoria
LUT	<i>Look-Up Table</i> – Tabla de búsqueda
MDM	<i>MicroBlaze Debug Module</i> – Módulo de depuración de MicroBlaze

MMIO	<i>Memory-Mapped I/O</i> – Entrada/salida mapeada en memoria
NCR	<i>Number of CRPs</i> – Número de pares desafío-respuesta requeridos para modelar una PUF
NLFSR	<i>Non-Linear Feedback Shift Register</i> – Registro de desplazamiento con retroalimentación no lineal
NLPUF	<i>Non-Linear PUF</i> – PUF basada en lógica no lineal
PLL	<i>Phase-Locked Loop</i> – Lazo de fase enganchada
PLPUF	<i>Pseudo-LFSR PUF</i> – PUF basada en pseudo-LFSR
PRNG	<i>Pseudo-Random Number Generator</i> – Generador de números pseudoaleatorios
PUF	<i>Physical Unclonable Function</i> – Función físicamente no clonable
RISC-V	<i>Reduced Instruction Set Computer V</i> – Arquitectura abierta de conjunto de instrucciones reducido
RO-PUF	<i>Ring Oscillator PUF</i> – PUF basada en oscilador de anillo
SoC	<i>System-on-Chip</i> – Sistema en chip
SRAM	<i>Static Random Access Memory</i> – Memoria estática de acceso aleatorio
TMR	<i>Triple Modular Redundancy</i> – Redundancia modular triple
TTM	<i>Time to Model</i> – Tiempo computacional para modelar una PUF
UART	<i>Universal Asynchronous Receiver-Transmitter</i> – Receptor y transmisor asíncrono universal
XDC	<i>Xilinx Design Constraints</i> – Archivo de restricciones de diseño
XOR	<i>Exclusive OR</i> – Compuerta lógica O-exclusiva
XSA	<i>Xilinx Support Archive</i> – Archivo de soporte de Xilinx

Capítulo 1

Introducción

La seguridad en los sistemas embebidos ha adquirido un papel fundamental debido al crecimiento acelerado del Internet de las Cosas (IoT), los dispositivos móviles y las infraestructuras industriales interconectadas. En este nuevo escenario tecnológico, millones de dispositivos operan de forma distribuida y, en muchos casos, en entornos físicamente accesibles, lo que incrementa significativamente la superficie de ataque [5]. Como consecuencia, garantizar la autenticidad, integridad y confiabilidad de cada dispositivo se convierte en un requisito esencial para prevenir clonación, falsificación y accesos no autorizados.

Tradicionalmente, la seguridad criptográfica se fundamenta en el uso de claves secretas almacenadas en memorias no volátiles. Sin embargo, este enfoque introduce una debilidad estructural: la información sensible permanece físicamente almacenada dentro del dispositivo, quedando expuesta a ataques invasivos y semi-invasivos capaces de extraer las claves directamente del hardware [6]. Esta problemática es especialmente crítica en plataformas embebidas de bajo costo y recursos limitados, donde la incorporación de mecanismos avanzados de protección física resulta compleja o económicamente inviable.

Ante esta limitación, las Funciones Físicamente No Clonables (PUFs, por sus siglas en inglés *Physical Unclonable Functions*) han emergido como una alternativa prometedora para el establecimiento de raíces de confianza en hardware. Las PUFs explotan las variaciones físicas inherentes al proceso de fabricación de los circuitos integrados para generar respuestas únicas e irrepetibles, eliminando la necesidad de almacenar claves criptográficas de forma permanente. Gracias a esta propiedad, las PUFs permiten implementar mecanismos de autenticación, identificación de dispositivos y generación dinámica de claves con un nivel elevado de resistencia frente a ataques físicos [5, 7].

Dentro de las múltiples arquitecturas existentes, la variante PLPUF (*Pseudo Linear Feedback Shift Register PUF*) representa una solución particularmente atractiva para plataformas reconfigurables. A diferencia de otras implementaciones basadas en retardos o elementos de memoria, el PLPUF emplea lógica combinatorial para generar respuestas multibit a partir de un único desafío, logrando una implementación compacta, eficiente en

área y altamente parametrizable [4]. Estas características lo convierten en un candidato idóneo para su integración en sistemas embebidos modernos basados en FPGA.

En este contexto, el presente trabajo propone el diseño, implementación e integración de un módulo PLPUF programable como periférico dentro de un *System-on-Chip* (SoC) basado en FPGA. El módulo se conecta mediante la interfaz estándar AXI y es controlado por el procesador suave MicroBlaze V sobre la plataforma Basys3 con FPGA Artix-7. Este enfoque elimina la dependencia de almacenamiento persistente de claves, fortaleciendo la seguridad a nivel físico, y provee una plataforma experimental para evaluar esquemas de autenticación y generación dinámica de claves en sistemas embebidos con recursos limitados.

De esta manera, el proyecto se posiciona en la intersección entre el diseño de sistemas embebidos y la seguridad de hardware, contribuyendo al desarrollo de soluciones abiertas, reconfigurables y reproducibles orientadas a fortalecer la seguridad desde el propio nivel físico del dispositivo.

1.1 Antecedentes del proyecto

1.1.1 Descripción de la institución

El Instituto Tecnológico de Costa Rica (TEC) es una institución pública de educación superior fundada en 1971 con el propósito de impulsar el desarrollo científico y tecnológico del país. A lo largo de su trayectoria, el TEC se ha consolidado como un referente nacional en formación profesional e investigación aplicada, destacándose por su enfoque en innovación tecnológica, excelencia académica y compromiso con el desarrollo sostenible [8].

El presente proyecto se desarrolla en la Escuela de Ingeniería en Computadores, unidad académica orientada a la integración de conocimientos en Electrónica y Ciencias de la Computación. Esta escuela promueve activamente la participación estudiantil en proyectos de investigación y desarrollo en áreas como diseño digital, sistemas embebidos, verificación de hardware, inteligencia artificial y seguridad informática.

1.1.2 Área de conocimiento del proyecto

El trabajo se enmarca dentro del área de Sistemas Embebidos, al involucrar el diseño e implementación de un módulo hardware sobre una arquitectura reconfigurable (FPGA) y su integración dentro de un System-on-Chip (SoC) con procesador y periféricos de comunicación.

Paralelamente, el proyecto se vincula con la Seguridad de Hardware mediante la implementación de una Función Físicamente No Clonable (PUF) como raíz de confianza,

orientada a la generación de identificadores únicos, protección de datos y autenticación de dispositivos sin requerir almacenamiento permanente de claves criptográficas.

1.1.3 Trabajos similares

La arquitectura Pseudo-LFSR PUF (PLPUF) fue propuesta por Hori et al. [4], quienes demostraron que una cadena de inversores con retroalimentación tipo Fibonacci, equivalente a un LFSR implementado de forma puramente combinacional, es capaz de generar respuestas multibit compactas y eficientes en área sobre FPGAs Xilinx Spartan. Su diseño alcanzó niveles competitivos de uniformidad y unicidad con un consumo reducido de recursos lógicos. Más recientemente, Alsharkawy et al. [9] analizaron los riesgos ocultos de la duración de activación en PLPUFs, evidenciando que una selección inadecuada de este parámetro puede comprometer la calidad de las respuestas y, por ende, la seguridad del sistema.

Diversas investigaciones han explorado variantes de PUFs basadas en registros de desplazamiento y lógica reconfigurable sobre FPGAs. Srilakshmi et al. [10] propusieron una PUF fuerte construida con un LFSR de bajo consumo, orientada a dispositivos con restricciones de energía. Hou et al. [6] presentaron una PUF LFSR ligera con seguridad mejorada implementada en FPGA, mientras que Huang et al. [11] extendieron el concepto hacia registros de desplazamiento no lineales (NLFSR PUF) para aumentar la resistencia frente a ataques de modelado por aprendizaje automático. Liu et al. [12] exploraron PUFs reconfigurables basadas en compuertas XOR como solución de bajo costo para seguridad en IoT. Las revisiones exhaustivas de Lata y Cenkeramaddi [13] y de Anandakumar et al. [14] ofrecen un panorama amplio del estado del arte en diseños de PUFs sobre FPGAs, abarcando desde arquitecturas basadas en retardos hasta implementaciones basadas en memoria.

En el ámbito de la integración de PUFs dentro de sistemas embebidos y protocolos de seguridad para IoT, Chen et al. [15] implementaron generadores de números pseudoaleatorios criptográficamente seguros basados en SRAM PUFs sobre FPGA. Idriss et al. [16] propusieron un protocolo de autenticación ligero basado en PUFs para dispositivos IoT con recursos limitados, mientras que Hou et al. [17] desarrollaron un protocolo de autenticación con doble PUF resistente a ataques de modelado físico. Estos trabajos evidencian una tendencia creciente hacia la incorporación de primitivas PUF como raíces de confianza en arquitecturas embebidas.

No obstante, a pesar de los avances descritos, no se identificó en la literatura un módulo PLPUF de código abierto, parametrizable y empaquetado como núcleo IP compatible con la interfaz AXI4-Lite y el procesador MicroBlaze V. La mayoría de las implementaciones reportadas se limitan a evaluaciones aisladas del núcleo PUF sin ofrecer integración dentro de un SoC completo ni controladores de software reutilizables. El presente proyecto aborda este vacío al proporcionar una solución integrada, desde el diseño HDL hasta el software de evaluación, orientada a entornos académicos y de investigación.

1.2 Planteamiento del problema

1.2.1 Contexto del problema

La implementación de sistemas seguros en plataformas reconfigurables, como las FPGAs, se ha convertido en una necesidad crítica frente al incremento de amenazas relacionadas con clonación de dispositivos, falsificación de hardware y accesos no autorizados. Los mecanismos criptográficos convencionales dependen de claves almacenadas en memorias no volátiles, tales como EEPROM, Flash o SRAM con respaldo de batería. Sin embargo, estas tecnologías no garantizan protección frente a ataques físicos capaces de extraer información sensible directamente del dispositivo [6].

Esta situación resulta especialmente crítica en el contexto del IoT y sistemas embebidos desplegados en entornos abiertos, donde los dispositivos operan con restricciones de energía, área y capacidad computacional. La incorporación de módulos adicionales de seguridad puede representar una sobrecarga significativa de recursos, dificultando la adopción de soluciones robustas [5, 18].

Durante la fabricación de circuitos integrados surgen variaciones físicas inevitables, tales como diferencias en el voltaje de umbral o el espesor del óxido de compuerta. Aunque estas variaciones no afectan la funcionalidad lógica del circuito, constituyen una fuente de entropía reproducible que puede explotarse para generar identificadores únicos de hardware [19].

1.2.2 Justificación del problema

Actualmente no se dispone de un módulo PUF de código abierto, programable y compatible con el procesador MicroBlaze V mediante la interfaz AXI. Muchas implementaciones reportadas en la literatura presentan limitaciones de accesibilidad, flexibilidad o integración dentro de arquitecturas modernas de sistemas embebidos, restringiendo su uso en entornos académicos y de investigación [13, 14]. Esta brecha es especialmente significativa en el ámbito educativo, donde las soluciones comerciales se encuentran protegidas por licencias o patentes que las hacen inaccesibles para instituciones formativas y estudiantes en formación.

El desarrollo de un módulo PLPUF programable e integrable directamente en un SoC permite eliminar la dependencia de almacenamiento persistente de claves, mejorar la seguridad a nivel físico y habilitar plataformas experimentales para la evaluación de técnicas criptográficas emergentes [20, 21]. A diferencia de implementaciones aisladas del núcleo PUF, una solución integrada como periférico AXI permite ejercitar simultáneamente todas las capas del sistema (HDL, interfaz de bus, *firmware*, comunicación serial y análisis de datos), proporcionando una plataforma realista para validar protocolos de autenticación basados en PUFs en condiciones equivalentes a un despliegue embebido real [16, 17].

Adicionalmente, la arquitectura PLPUF resulta especialmente atractiva para este propósito debido a su compacidad y a la posibilidad de modificar la relación desafío-respuesta mediante un único parámetro (la duración de activación), lo cual la convierte en una primitiva versátil para experimentar con distintos compromisos entre confiabilidad y entropía [4, 9]. Finalmente, el desarrollo de este módulo facilita su adopción en contextos educativos y prototipos de investigación dentro del ámbito nacional, contribuyendo a la formación de ingenieros capacitados en el diseño de sistemas embebidos seguros, área estratégica para el desarrollo tecnológico del país.

1.2.3 Enunciado del problema

Existe una vulnerabilidad crítica en sistemas embebidos basados en FPGA con procesadores suaves debido a la dependencia de claves criptográficas almacenadas en memorias no volátiles, las cuales pueden ser extraídas mediante ataques físicos. Esta vulnerabilidad se ve agravada por la ausencia de módulos de seguridad de hardware abiertos, programables y fácilmente integrables mediante estándares como AXI en arquitecturas basadas en MicroBlaze V dentro de FPGAs de la familia Artix-7.

1.3 Objetivos del proyecto

1.3.1 Objetivo general

Diseñar un módulo PLPUF programable compatible con el procesador MicroBlaze V y la interfaz AXI, mediante lenguajes de descripción de hardware y herramientas de síntesis en FPGA, para su integración en sistemas embebidos orientados a la seguridad.

1.3.2 Objetivos específicos

1. Implementar el módulo PLPUF en HDL con ancho parametrizable que genere identificadores únicos aprovechando variaciones físicas del hardware mediante síntesis en FPGA con AMD Vivado.
2. Desarrollar una interfaz AXI-*slave* que permita el control del módulo desde el procesador MicroBlaze V mediante registros de control y estado.
3. Integrar el módulo dentro de un sistema embebido completo con MicroBlaze V, memoria y UART, utilizando AMD Vitis para el desarrollo del software de control.
4. Evaluar el PLPUF mediante métricas de seguridad como uniformidad, unicidad y confiabilidad utilizando pruebas automatizadas de desafío-respuesta y análisis estadístico.

1.4 Alcances, entregables y limitaciones

1.4.1 Alcances

El proyecto comprende el diseño, implementación e integración de un módulo PLPUF programable en la placa Basys3 con FPGA Artix-7 (XC7A35T-1CPG236C). El módulo será incorporado como periférico AXI-*slave* dentro de un sistema SoC con procesador MicroBlaze V, memoria local y comunicación UART. Se desarrollará además el firmware de control en lenguaje C utilizando AMD Vitis y se realizará la evaluación estadística del módulo mediante métricas estándar de PUFs: uniformidad, unicidad y confiabilidad [22, 23].

1.4.2 Entregables

1. Código HDL del módulo PLPUF parametrizable con reportes de síntesis.
2. Módulo AXI-*slave* funcional con documentación del mapa de registros.
3. Sistema SoC completo con diseño en Vivado, *bitstream* y software de control.
4. *Dataset* de pares desafío-respuesta e informe de evaluación estadística.

1.4.3 Limitaciones

- El diseño está condicionado por los recursos disponibles en la FPGA Artix-7 asignada, lo que puede limitar parámetros como el ancho del PLPUF.
- Las herramientas utilizadas serán exclusivamente AMD Vivado y AMD Vitis.
- La evaluación estadística se realizará sobre un único dispositivo FPGA, por lo que los resultados pueden no generalizarse a otros lotes de fabricación [23].

Capítulo 2

Marco de referencia teórico

En este capítulo se presentan los fundamentos teóricos y tecnológicos que sustentan el diseño, la implementación y la evaluación del módulo PLPUF desarrollado en este trabajo. Se describen, en primer lugar, los conceptos de *System-on-Chip* (SoC) y las interfaces de comunicación empleadas; a continuación, la plataforma FPGA y las herramientas de desarrollo; luego, la teoría de las Funciones Físicamente No Clonables (PUFs) con énfasis en la variante Pseudo-LFSR; y, finalmente, las métricas de seguridad utilizadas para la evaluación del módulo.

2.1 Sistemas en chip (SoC)

Un *System-on-Chip* (SoC) es una arquitectura de circuito integrado que incorpora en un solo dispositivo un procesador, memoria, periféricos de comunicación y lógica personalizada. En el contexto de las FPGAs, los SoC se construyen instanciando procesadores suaves (*soft processors*) dentro de la lógica programable, conectados a periféricos mediante buses estándar [24]. Esta aproximación permite crear sistemas embebidos completos en los que tanto el hardware como el software son altamente configurables.

2.1.1 Arquitectura MicroBlaze V

MicroBlaze V es un procesador suave (*soft processor*) de AMD diseñado para su implementación en FPGAs de la familia AMD/Xilinx. A diferencia de su predecesor MicroBlaze, que utiliza un conjunto de instrucciones propietario, MicroBlaze V implementa la arquitectura de conjunto de instrucciones abierta RISC-V, específicamente el conjunto base RV32I [1]. RISC-V es una ISA de estándar abierto mantenida por RISC-V International, lo que permite a cualquier fabricante desarrollar procesadores compatibles sin costos de licenciamiento [1].

El conjunto fijo de características de MicroBlaze V incluye 32 registros de propósito

general, un contador de programa extensible de 32 bits, un *pipeline* de emisión única, una unidad aritmético-lógica (ALU), un *barrel shifter* y las extensiones **Zicsr** (instrucciones de registros de control y estado) y **Zifencei** (barrera de instrucciones) [1].

Adicionalmente, MicroBlaze V es altamente configurable mediante parámetros que permiten habilitar selectivamente funcionalidades opcionales. Entre las extensiones configurables se encuentran: **M** (multiplicación y división entera), **A** (instrucciones atómicas), **C** (instrucciones comprimidas de 16 bits), **F** (punto flotante de precisión simple), **D** (punto flotante de doble precisión) y **Zb[abcs]** (manipulación de bits) [1]. La profundidad del *pipeline* es seleccionable entre 3, 4, 5 y 8 etapas, lo que permite optimizar el diseño para área o frecuencia según la aplicación [1].

En cuanto a la interfaz de memoria, MicroBlaze V ofrece un bus local de memoria (LMB) para acceso de baja latencia a BRAM, e interfaces AXI4 para cachés de instrucciones y datos (**M_AXI_IC**, **M_AXI_DC**), así como un puerto de datos de periféricos (**M_AXI_DP**) que soporta el protocolo AXI4-Lite [1, 25]. El procesador también integra opcionalmente cachés de instrucciones y datos con políticas configurables (*write-back* o *write-through*), soporte de depuración mediante el módulo MDM V (*MicroBlaze Debug Module V*), y capacidades de tolerancia a fallos como operación en *lockstep* y redundancia modular triple (TMR) [1].

Como se muestra en la Figura 2.1, la arquitectura interna de MicroBlaze V se organiza en torno a un decodificador de instrucciones central que alimenta la ALU, el *barrel shifter*, el multiplicador y el divisor (estos dos últimos opcionales). El banco de 32 registros de propósito general y los registros de control y estado (CSR) están conectados directamente al decodificador. En el lado de instrucciones, un búfer de instrucciones recibe datos desde el bus local de instrucciones (ILMB) o desde la caché de instrucciones (**M_AXI_IC**); un decodificador de instrucciones comprimidas (extensión **C**) procesa opcionalmente las instrucciones de 16 bits. En el lado de datos, el procesador accede a la BRAM mediante el bus local de datos (DLMB) y a los periféricos mediante el puerto AXI de datos (**M_AXI_DP**); la caché de datos (**M_AXI_DC**) es configurable como *write-back* o *write-through*. La unidad de punto flotante (FPU), la unidad de gestión de memoria (MMU con TLBs) y la caché de predicción de saltos (*Branch Target Cache*) aparecen sombreadas en el diagrama, indicando que son características opcionales habilitables mediante parámetros de configuración.

MicroBlaze V es totalmente compatible a nivel de hardware con el procesador MicroBlaze clásico [1], lo que permite reutilizar la biblioteca de periféricos AXI existente del ecosistema AMD/Xilinx. La utilización de recursos en una configuración básica RV32I optimizada para área requiere aproximadamente 633 LUTs en una FPGA Spartan-7, mientras que una configuración RV32IMAFc con extensiones de manipulación de bits alcanza aproximadamente 4826 LUTs [1].

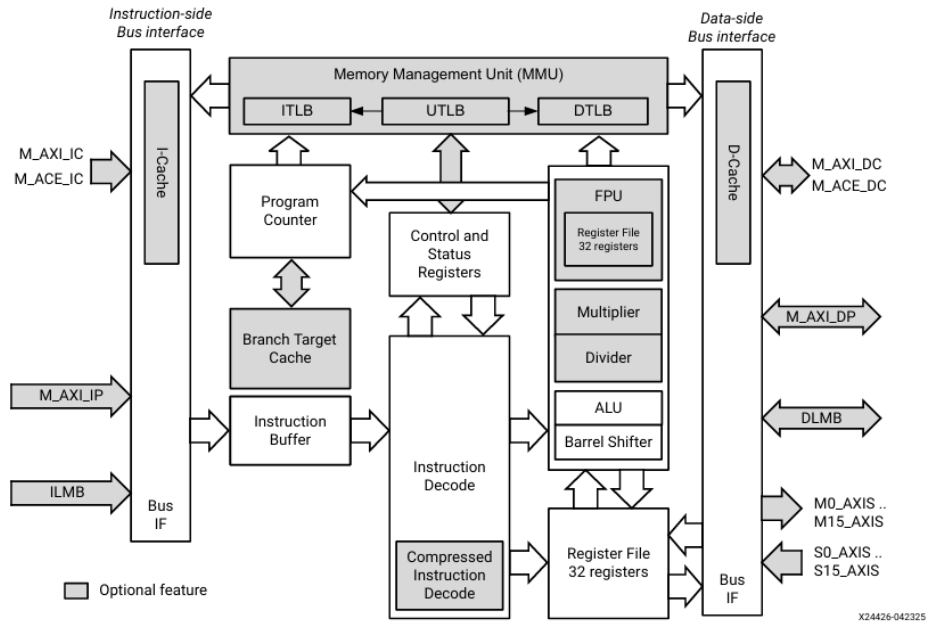


Figura 2.1: Diagrama de bloques del procesador MicroBlaze V [1].

2.1.2 Bus AXI4-Lite

La interfaz AMBA AXI (*Advanced eXtensible Interface*) es un protocolo de comunicación *on-chip* desarrollado por Arm Ltd. como parte de la especificación AMBA (*Advanced Microcontroller Bus Architecture*) [2]. AMD/Xilinx adoptó AXI como el protocolo estándar de interconexión para sus FPGAs a partir de la serie 7, reemplazando los buses PLB y FSL utilizados en generaciones anteriores [26].

El protocolo AXI define cinco canales independientes que operan de forma desacoplada mediante un mecanismo de *handshake* basado en señales **VALID** y **READY** [2]. La Figura 2.2 ilustra los tres canales involucrados en una transacción de escritura: el canal de dirección de escritura (AW), por el que la interfaz maestra (*Manager*) envía la dirección y las señales de control; el canal de datos de escritura (W), que transporta las palabras de datos desde el maestro hacia la interfaz subordinada (*Subordinate*); y el canal de respuesta de escritura (B), por el que el subordinado retorna el estado de finalización de la transacción. De forma análoga, la Figura 2.3 muestra los dos canales de lectura: el canal de dirección de lectura (AR), por el que el maestro emite la dirección solicitada, y el canal de datos de lectura (R), por el que el subordinado retorna los datos leídos junto con su estado. Las flechas en ambos diagramas indican la dirección del flujo de información entre las interfaces.

La variante AXI4-Lite es una simplificación del protocolo AXI4 completo orientada a periféricos de control con mapas de registros accesibles mediante *memory-mapped I/O* (MMIO). AXI4-Lite restringe las transacciones a transferencias de una sola palabra (sin ráfagas) y elimina señales avanzadas como **AWLEN**, **AWSIZE** y **AWBURST**, lo que simplifica significativamente la lógica del esclavo [2, 26]. Cada transacción se completa cuando el

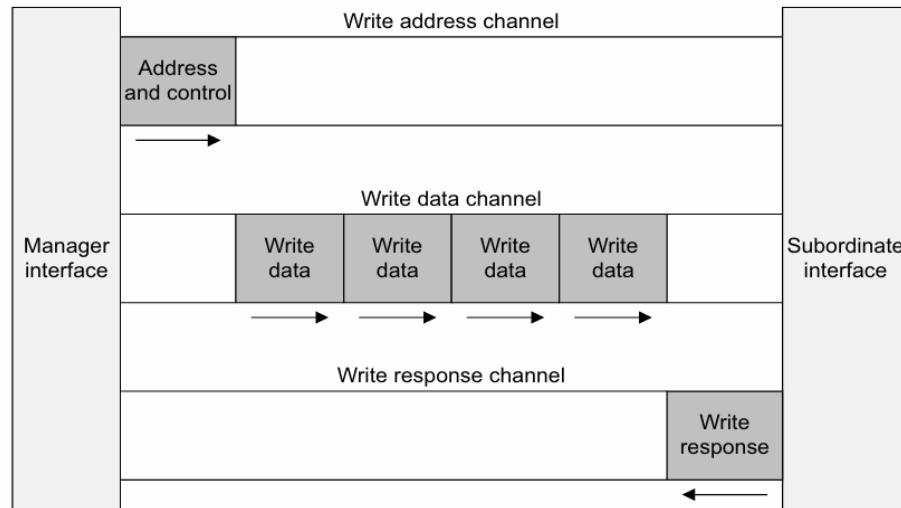


Figura 2.2: Canales de escritura del protocolo AXI: canal de dirección (AW), canal de datos (W) y canal de respuesta (B) [2].

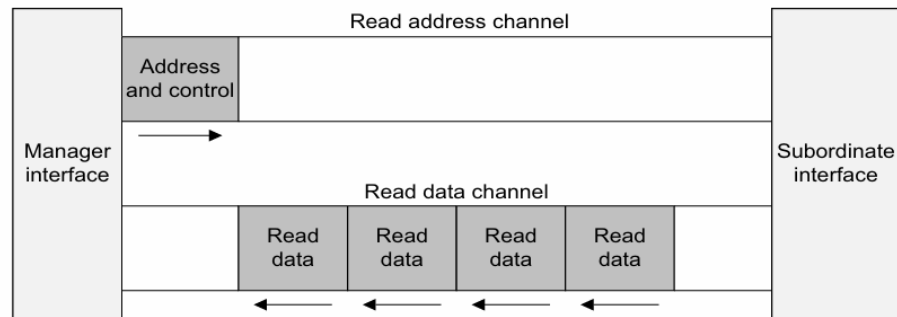


Figura 2.3: Canales de lectura del protocolo AXI: canal de dirección (AR) y canal de datos (R) [2].

emisor activa **VALID** y el receptor responde con **READY** en el mismo ciclo de reloj.

En el ecosistema Vivado, los periféricos AXI4-Lite se integran en el *block design* mediante el bloque SmartConnect, que actúa como interconexión entre el puerto maestro del procesador (**M_AXI_DP**) y los puertos esclavos de los periféricos. El entorno IP Integrator automatiza la asignación de direcciones y la conexión de señales de reloj y *reset* [24, 26].

2.2 FPGAs y lógica reconfigurable

Las FPGAs (*Field Programmable Gate Arrays*) son dispositivos semiconductores cuya funcionalidad lógica puede ser programada y reprogramada por el usuario después de su fabricación. A diferencia de los circuitos integrados de aplicación específica (ASICs), las FPGAs ofrecen flexibilidad de diseño, menor tiempo de desarrollo y la posibilidad

de actualización en campo, lo que las convierte en plataformas idóneas para prototipado rápido, sistemas embebidos y aplicaciones de seguridad [13, 14].

La naturaleza reconfigurable de las FPGAs resulta particularmente relevante para la implementación de PUFs, ya que permite explotar las variaciones físicas inherentes a la lógica programable, como diferencias en los retardos de propagación de LUTs e interconexiones, para generar identificadores únicos de dispositivo [13].

2.2.1 Familia Artix-7 y plataforma Basys 3

La familia Artix-7 de AMD/Xilinx forma parte de la serie 7 de FPGAs y está optimizada para aplicaciones que requieren alto rendimiento lógico con bajo consumo de energía. El presente proyecto utiliza la placa de desarrollo Basys 3 de Digilent, que integra una FPGA Artix-7 con número de parte XC7A35T-1CPG236C [3].

Las principales características del dispositivo XC7A35T son [3]:

- 33 280 celdas lógicas distribuidas en 5 200 *slices*, donde cada *slice* contiene cuatro LUTs de 6 entradas y 8 *flip-flops*.
- 1 800 Kb de memoria RAM de bloques (BRAM).
- 5 *tiles* de gestión de reloj, cada uno con un lazo de fase enganchada (PLL).
- 90 bloques DSP para operaciones aritméticas aceleradas.
- Frecuencias internas de reloj superiores a 450 MHz.
- Conversor analógico-digital integrado (XADC).

La placa Basys 3 complementa la FPGA con periféricos de entrada/salida que facilitan la validación de diseños: 16 interruptores, 16 LEDs, 5 botones pulsadores, un *display* de 7 segmentos de 4 dígitos, un puerto USB-UART para comunicación serial, conectores Pmod para expansión y un puerto VGA de 12 bits [3]. El reloj principal del sistema es un oscilador de 100 MHz conectado al pin W5 de la FPGA [3].

La Figura 2.4 muestra la disposición física de la placa Basys 3 con sus componentes numerados. En la parte central se encuentra el chip FPGA Artix-7, rodeado por los periféricos de entrada/salida: los 16 interruptores y los 16 LEDs se ubican en el borde inferior de la placa; los 5 pulsadores se sitúan a la derecha junto a los conectores Pmod para expansión; el *display* de 7 segmentos de 4 dígitos se encuentra en la zona inferior izquierda. En el borde superior se localizan el conector VGA, el puerto USB compartido para UART y JTAG (marcado como *Shared UART/JTAG USB port*), el conector USB *host* y los controles de alimentación. El puerto USB-UART compartido (conector J4) es el que se utiliza en este proyecto tanto para la carga del *bitstream* vía JTAG como para la comunicación serial con el computador anfitrión.

La configuración de la FPGA se realiza mediante archivos *bitstream* generados por Vivado. El *bitstream* del XC7A35T tiene un tamaño típico de 17 536 096 bits y puede cargarse a través de JTAG (puerto USB J4), desde la memoria Flash SPI integrada o desde una memoria USB conectada al puerto HID [3].

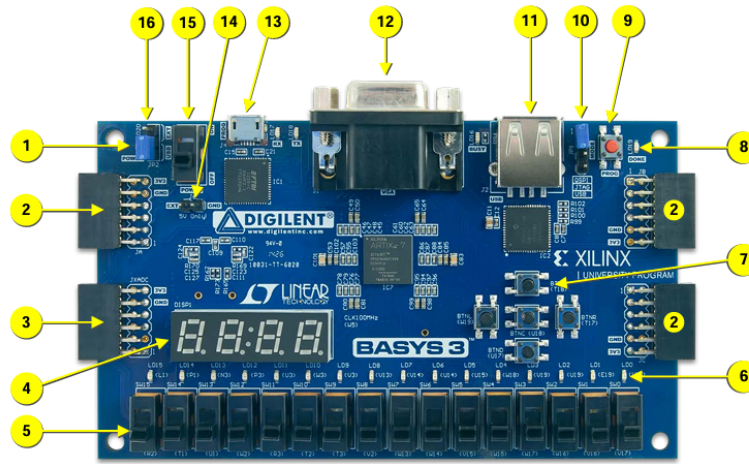


Figure 1. Basys3 FPGA board with callouts.

Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod connector(s)	10	Programming mode jumper
3	Analog signal Pmod connector (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/ JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

Table 1. Basys3 Callouts and component descriptions.

Figura 2.4: Placa de desarrollo Basys 3 con FPGA Artix-7 [3].

2.2.2 Herramientas: Vivado y Vitis

El flujo de desarrollo de sistemas embebidos sobre FPGAs de AMD/Xilinx se sustenta en dos herramientas complementarias: Vivado Design Suite para el diseño de hardware y Vitis Unified Software Platform para el desarrollo de software.

AMD Vivado Design Suite es el entorno de diseño para FPGAs de la serie 7 en adelante. Vivado incluye el módulo IP Integrator, que permite construir diseños de bloques (*block designs*) mediante la interconexión gráfica de núcleos IP (*Intellectual Property*), tales como procesadores, controladores de memoria, interfaces de comunicación y periféricos personalizados [24]. El flujo típico de un proyecto en Vivado comprende: creación del *block design*, generación de productos de salida de los IPs, síntesis lógica, implementación física (*place and route*), verificación de restricciones temporales y generación del *bitstream* [24]. Para integrar periféricos personalizados, como el módulo PLPUF de este proyecto, Vivado ofrece un flujo de empaquetado de IP que genera la descripción IP-XACT (*component.xml*), permitiendo que el periférico sea reutilizable en cualquier *block design* [26].

AMD Vitis es la plataforma unificada de desarrollo de software para sistemas embebidos basados en procesadores AMD. Vitis importa la descripción del hardware exportada por Vivado en formato `.xsa` (*Xilinx Support Archive*), a partir de la cual genera automáticamente la plataforma de software, incluyendo el *Board Support Package* (BSP) con controladores de periféricos y la biblioteca estándar de C [27]. El desarrollador crea aplicaciones en C/C++ que se compilan con la cadena de herramientas GCC para RISC-V (`riscv64-unknown-elf-gcc`) y se cargan en la FPGA a través de JTAG para su ejecución y depuración [27].

Este flujo HW–SW permite un desarrollo iterativo donde las modificaciones al hardware (por ejemplo, cambios en el mapa de registros del periférico PLPUF) se propagan al software mediante la regeneración del archivo `.xsa` y la actualización de la plataforma en Vitis [24, 27].

2.3 Funciones físicamente no clonables (PUF)

Las Funciones Físicamente No Clonables (PUFs, por sus siglas en inglés *Physical Unclo-nable Functions*) son primitivas de seguridad de hardware que explotan las variaciones microscópicas e incontrolables introducidas durante el proceso de fabricación de circuitos integrados para generar identificadores únicos y respuestas criptográficamente útiles [5, 7]. Estas variaciones, como diferencias en el voltaje de umbral de los transistores, el espesor del óxido de compuerta o los retardos de propagación en las interconexiones, constituyen una fuente natural de entropía que es reproducible dentro del mismo dispositivo pero impredecible e imposible de replicar exactamente en otro chip, incluso del mismo lote de fabricación [14, 19].

A diferencia de los mecanismos criptográficos convencionales, que dependen de claves almacenadas en memorias no volátiles (EEPROM, Flash, SRAM con respaldo de batería), las PUFs generan sus claves directamente desde las propiedades físicas del silicio, eliminando la necesidad de almacenamiento permanente de información sensible [6, 7]. Esta propiedad las hace intrínsecamente resistentes a ataques físicos de extracción de claves y las posiciona como una alternativa prometedora para el establecimiento de raíces de confianza en hardware [5].

2.3.1 Fundamentos y taxonomía

Una PUF se define formalmente como una función implementada por un sistema físico que es fácil de evaluar pero extremadamente difícil o imposible de clonar [14]. La interacción con una PUF se realiza mediante un mecanismo de desafío-respuesta: ante un estímulo de entrada (desafío), la PUF produce una salida (respuesta) que depende de las características físicas únicas del dispositivo. Las propiedades esenciales que debe exhibir una PUF son [13, 14]:

- **Reproducibilidad:** la respuesta a un desafío dado debe ser consistente a lo largo del tiempo y bajo diferentes condiciones ambientales.
- **Unicidad:** diferentes dispositivos deben producir respuestas significativamente distintas ante el mismo desafío.
- **Inclonabilidad:** debe ser computacional y físicamente inviable construir una réplica del dispositivo o un modelo matemático que reproduzca fielmente el comportamiento de la PUF.
- **Impredecibilidad:** no debe ser posible predecir la respuesta a un desafío no observado previamente.
- **Evidencia de manipulación (*tamper-evidence*):** cualquier alteración física del circuito debe modificar las respuestas de forma detectable.

Las variaciones del proceso de fabricación que explotan las PUFs se clasifican en dos categorías [14]: variaciones *die-to-die*, que afectan a todo el chip de manera uniforme (inter-oblea, inter-lote); y variaciones *within-die*, que ocurren entre diferentes elementos del mismo chip. Dentro de estas últimas, las variaciones aleatorias, causadas por la naturaleza estocástica del dopado y el crecimiento del óxido de compuerta, son la fuente principal de entropía y de la propiedad de inclonabilidad de las PUFs [14, 19].

Las PUFs se clasifican generalmente según el tamaño de su espacio de pares desafío-respuesta (CRP) en dos categorías principales [5, 13]:

- **PUF débil (*Weak PUF*):** genera un número limitado de CRPs, típicamente uno por instancia. Se utiliza principalmente para generación de claves criptográficas y almacenamiento seguro de claves. Ejemplos representativos incluyen las PUFs basadas en SRAM y *flip-flop* [13, 15].
- **PUF fuerte (*Strong PUF*):** posee un espacio de CRPs exponencialmente grande en relación con su tamaño de implementación, lo que dificulta que un atacante capture todos los pares. Es adecuada para protocolos de autenticación donde cada CRP se utiliza una sola vez. Ejemplos incluyen la Arbiter PUF y la PUF de oscilador de anillo (RO-PUF) [5, 13].

Desde la perspectiva de la arquitectura del circuito, las PUFs basadas en FPGA pueden clasificarse en tres familias principales [13, 14]:

1. **PUFs basadas en retardo:** explotan las diferencias en los tiempos de propagación de señales a través de caminos lógicos nominalmente idénticos. Incluyen la Arbiter PUF, la RO-PUF y la Pseudo-LFSR PUF (PLPUF) [4].
2. **PUFs basadas en memoria:** aprovechan el estado de encendido aleatorio de elementos de memoria biestables, como celdas SRAM o *flip-flops* [15].

3. **PUFs híbridas y reconfigurables:** combinan técnicas de las dos familias anteriores o incorporan mecanismos de reconfiguración para ampliar el espacio de CRPs [12].

2.3.2 PUFs programables: Pseudo-LFSR PUF (PLPUF)

La Pseudo-LFSR PUF (PL-PUF o PLPUF) es una arquitectura de PUF propuesta por Hori et al. [4] que se basa en la estructura de un registro de desplazamiento con retroalimentación lineal (LFSR), pero a diferencia de un LFSR convencional, **no contiene registros de desplazamiento**. En su lugar, el PLPUF emplea una cadena de lógica puramente combinacional compuesta por inversores y compuertas XOR dispuestos en una topología de retroalimentación tipo Fibonacci [4].

Arquitectura del núcleo

Para un PLPUF de n bits, el diseño requiere únicamente n inversores y un número reducido de compuertas XOR determinado por el polinomio primitivo utilizado. En la configuración de 128 bits propuesta por [4], el polinomio primitivo empleado es:

$$p(x) = x^{128} + x^{126} + x^{102} + x^{99} + 1 \quad (2.1)$$

lo que implica solamente tres compuertas XOR en las posiciones de *tap* correspondientes (índices 125, 101 y 98, indexados desde cero) [4]. Cada celda del PLPUF contiene un multiplexor que selecciona entre el bit de desafío correspondiente (durante la fase de carga) o la señal de retroalimentación del vecino (durante la fase activa), y un inversor cuyo retardo de propagación constituye la fuente de entropía física. Los inversores llevan el atributo DONT_TOUCH para evitar que las herramientas de síntesis optimicen o eliminen el lazo combinacional.

La Figura 2.5 presenta la estructura completa del PLPUF de 128 bits. La cadena está formada por celdas de lógica central (*Core logic*) conectadas en serie: la salida Dout [128] alimenta la entrada de la siguiente celda, y así sucesivamente hasta Dout [1]. La línea de retroalimentación conecta Dout [1] de vuelta a la entrada de la primera celda, cerrando el lazo. En las posiciones de *tap* (celdas 127, 102 y 100, correspondientes a los términos del polinomio primitivo) se insertan compuertas XOR ($\oplus$) que combinan la señal de retroalimentación con la salida del vecino adyacente, introduciendo la no linealidad necesaria para la difusión de la entropía.

La Figura 2.6 detalla la estructura interna de una celda individual. Cada celda recibe dos entradas: Din (señal de retroalimentación del vecino durante la fase activa) y Dinit (bit de desafío correspondiente durante la fase de carga). La señal de selección SEL controla un multiplexor que elige entre ambas entradas. La señal seleccionada pasa a través de un inversor, cuyo retardo de propagación (determinado por las variaciones del proceso de fabricación) constituye la fuente de entropía física. La salida Dout alimenta la siguiente celda de la cadena.

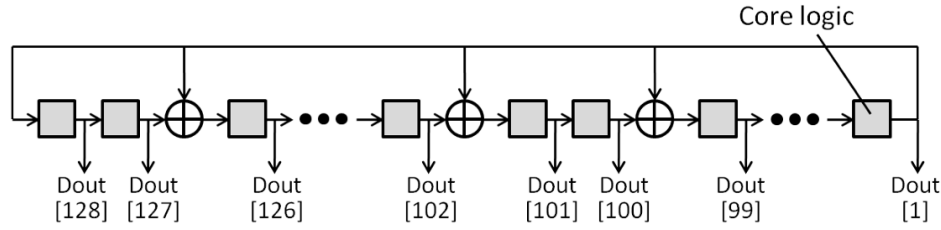


Figura 2.5: Estructura del Pseudo-LFSR PUF de 128 bits: cadena de celdas de lógica central con compuertas XOR en las posiciones de *tap* y retroalimentación desde Dout [1] hacia la primera celda [4].

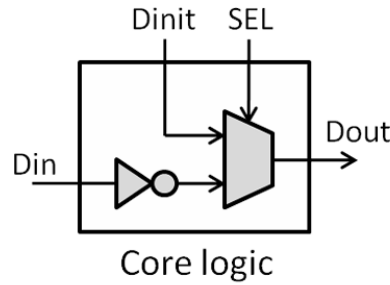


Figura 2.6: Estructura interna de una celda de lógica central: multiplexor controlado por SEL e inversor como fuente de entropía [4].

Operación del PLPUF

El funcionamiento del PLPUF se controla mediante una máquina de estados finitos (FSM) con cuatro estados, ilustrada en la Figura 2.7 [4]. El diagrama muestra las transiciones entre estados y las señales de control asociadas a cada uno:

1. **IDLE** (`enable = 0`, `busy = 0`, `done = 0`): el módulo está inactivo, esperando la señal de inicio. Al recibir `start = 1`, la FSM activa `busy = 1` y transiciona a **LOAD**.
2. **LOAD** (1 ciclo): el desafío de n bits se propaga a través de los inversores con la señal de habilitación desactivada (`enable = 0`), permitiendo que la cadena se estabilice en un estado determinado por el desafío. Como se observa en la figura, en este estado el desafío (`challenge`) se propaga por la cadena mientras `enable` permanece en cero. Esta fase dura un solo ciclo de reloj, tras lo cual `enable` pasa a 1 y la FSM avanza a **ACTIVE**.
3. **ACTIVE** (`enable = 1`): el lazo de retroalimentación se cierra y el circuito oscila libremente a una frecuencia determinada por los retardos acumulados de propagación de cada inversor y compuerta XOR, retardos que son únicos para cada chip fabricado. Esta fase dura exactamente c ciclos de reloj, donde c es la **duración de activación** configurable por software. Cuando el contador descendente alcanza `duration - 1`, el estado de la cadena se captura en el registro de respuesta (`response ← core_out`).

y la FSM transiciona a DONE.

4. **DONE** (1 ciclo, $\text{done} = 1$, $\text{busy} = 0$): la señal **done** emite un pulso de un ciclo, indicando que la respuesta de n bits está disponible en los registros de salida. La FSM retorna automáticamente a **IDLE**, quedando lista para una nueva evaluación.

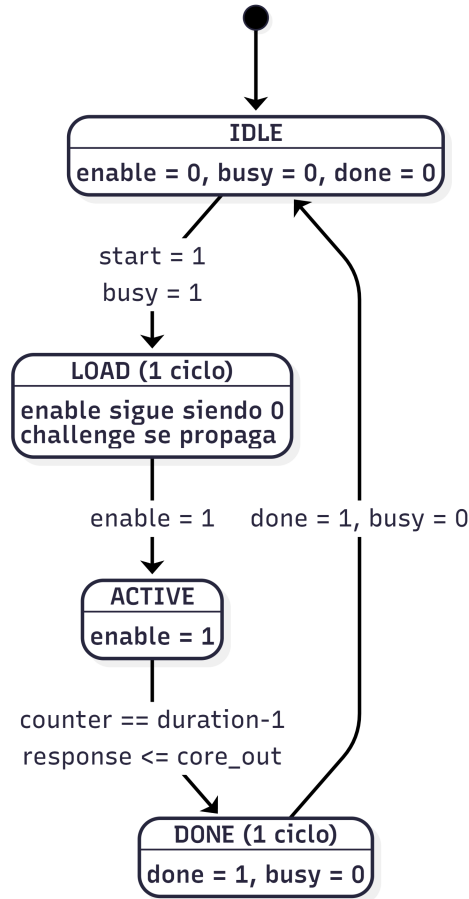


Figura 2.7: Máquina de estados del PLPUF: ciclo de evaluación desafío-respuesta con las señales de control y las condiciones de transición entre los estados **IDLE**, **LOAD**, **ACTIVE** y **DONE**.

Propiedades distintivas

El PLPUF presenta varias propiedades que lo distinguen de otras arquitecturas de PUF [4]:

- **Compacidad:** requiere solo n inversores y un número reducido de compuertas XOR, en contraste con la Arbiter PUF que necesita $2n$ multiplexores para un diseño de n bits.
- **Eficiencia:** genera una respuesta de n bits a partir de un solo desafío de n bits en una única evaluación.

- **Multifuncionalidad:** la relación desafío-respuesta puede modificarse sin alterar el hardware, simplemente variando la duración de activación c . Cada valor de c genera un mapeo desafío-respuesta diferente, lo que equivale conceptualmente a disponer de múltiples núcleos PUF dentro de un solo circuito.
- **Configurabilidad de confiabilidad:** la confiabilidad de las respuestas puede ajustarse seleccionando un valor adecuado de c . Con duraciones cortas ($c < 4$), el PLPUF exhibe una distancia de Hamming intra-dispositivo baja (alta confiabilidad); a medida que c aumenta, la variabilidad de las respuestas crece [4].

Resultados experimentales de referencia

Hori et al. [4] implementaron un PLPUF de 128 bits en 16 FPGAs Virtex-5 XC5VLX30 montadas en placas SASEBO-GII, operando a 24 MHz con tensión de núcleo de 1,0 V. Se evaluaron 100 desafíos aleatorios con 100 repeticiones cada uno para valores de duración de activación $c = 1, \dots, 16$. Los resultados mostraron una distancia de Hamming intra-dispositivo pequeña para valores bajos de c (por ejemplo, media de 1,76 bits para $c = 1$) y tasas de falsa aceptación y falso rechazo (FAR/FRR) iguales a cero para múltiples dispositivos con $c \leq 4$ [4].

Riesgos de la duración de activación

Investigaciones recientes de Alsharkawy et al. [9] demostraron que la duración de activación constituye un parámetro crítico de seguridad en los PLPUFs. Mediante un análisis de dependencia lineal entre pares de PLPUFs implementados en 8 FPGAs ARTY A7 (32 instancias, 8 192 CRPs cada una), los autores encontraron que con $c = 1$ la predictibilidad promedio entre PLPUFs ubicados en la misma posición física alcanza hasta el 96 %, mientras que con $c = 31$ la predictibilidad disminuye a aproximadamente 30 % [9]. Este hallazgo subraya la importancia de seleccionar cuidadosamente la duración de activación para equilibrar confiabilidad y resistencia ante ataques predictivos.

2.3.3 Modelos de desafío-respuesta (CRP)

El modelo de desafío-respuesta (*Challenge-Response Pair*, CRP) es el paradigma fundamental de interacción con una PUF. Un desafío C es un estímulo de entrada de n bits que se aplica al circuito PUF, y la respuesta R es la salida de n bits producida por la PUF en función de las características físicas únicas del dispositivo [5]. Cada dispositivo fabricado genera un conjunto de CRPs propio y diferente, que actúa como una huella digital del hardware.

El espacio total de CRPs de una PUF determina su clasificación funcional y sus aplicaciones de seguridad [5, 18]:

- En una **PUF débil**, el espacio de CRPs es limitado (a menudo un solo par), por lo que se emplea para generación de claves y almacenamiento seguro de secretos.
- En una **PUF fuerte**, el espacio crece exponencialmente con el tamaño de la implementación (2^n para n bits de desafío), lo que permite protocolos de autenticación donde cada CRP se utiliza una sola vez para evitar ataques de reproducción (*replay attacks*) [5].

En el caso específico del PLPUF, la multifuncionalidad proporcionada por la duración de activación c amplía efectivamente el espacio de CRPs: cada combinación de desafío y duración (C, c) produce una respuesta diferente, de modo que un PLPUF de n bits con un rango de duraciones de 1 a $c_{\max}$ ofrece hasta $c_{\max} \times 2^n$ pares desafío-respuesta distintos [4].

La seguridad de un esquema basado en CRPs puede evaluarse mediante dos métricas complementarias [28]: el número de CRPs que un atacante debe recopilar para modelar exitosamente la PUF con una predictibilidad del 99 % (*NCR*, *Number of CRPs*); y el tiempo computacional requerido para construir dicho modelo (*TTM*, *Time to Model*). Una PUF es considerada segura cuando ambas métricas exceden los límites prácticos de un atacante.

2.4 Seguridad en sistemas embebidos

La seguridad de los sistemas embebidos enfrenta desafíos crecientes debido a la proliferación de dispositivos IoT en entornos físicamente accesibles, donde los atacantes pueden realizar ataques invasivos y semi-invasivos sobre el hardware [5, 29]. En este contexto, las PUFs ofrecen una alternativa de bajo costo y recursos reducidos para establecer raíces de confianza directamente en el silicio, sin depender de almacenamiento criptográfico externo [7].

2.4.1 Autenticación de hardware

La autenticación de hardware basada en PUFs sigue un esquema de dos fases: registro (*enrollment*) y verificación [5, 30].

1. **Fase de registro:** en un entorno controlado y seguro, se aplican al dispositivo un conjunto de desafíos $\{C_1, C_2, \dots, C_m\}$ y se almacenan las respuestas correspondientes $\{R_1, R_2, \dots, R_m\}$ en una base de datos del servidor verificador. Dado que las PUFs no son perfectamente estables, se puede emplear votación por mayoría sobre múltiples evaluaciones para obtener respuestas de referencia (*golden responses*) [22].
2. **Fase de verificación:** el servidor envía un desafío C_i al dispositivo y compara la respuesta recibida R'_i con la respuesta de referencia R_i almacenada. Si la distancia

de Hamming entre ambas es menor que un umbral predefinido t , el dispositivo se considera auténtico [5].

Para aplicaciones que requieren generación de claves criptográficas estables a partir de respuestas PUF ruidosas, se emplean algoritmos de datos auxiliares (*Helper Data Algorithms*, HDA) que combinan la respuesta de la PUF con datos públicos y códigos correctores de errores para reproducir una clave determinística [20]. Este enfoque permite derivar claves de hasta 128 o 256 bits con alta confiabilidad, sin almacenar la clave en sí misma en memoria no volátil [20, 21].

La resistencia de las PUFs a la computación cuántica constituye una ventaja adicional: dado que la seguridad de las PUFs no depende de la dificultad de problemas matemáticos (como la factorización de enteros o el logaritmo discreto), sino de propiedades físicas del silicio, se consideran intrínsecamente resistentes a ataques cuánticos que amenazan los algoritmos criptográficos convencionales [5, 30].

2.4.2 Métricas de evaluación: unicidad, confiabilidad y uniformidad

La evaluación del rendimiento de una PUF requiere un conjunto de métricas estadísticas bien definidas. Maiti et al. [22] propusieron un marco sistemático basado en siete parámetros agrupados en tres dimensiones: dispositivo (variabilidad inter-chip), tiempo (estabilidad intra-chip) y espacio (distribución de bits). A continuación se describen las métricas principales empleadas en este trabajo.

Uniformidad

La uniformidad mide la distribución de bits en la respuesta de una PUF. Para una PUF ideal, la proporción de unos en la respuesta debe ser del 50 %, indicando máxima entropía por bit [22]. Se calcula como:

$$\text{Uniformidad} = \frac{1}{n} \sum_{j=1}^n r_j \times 100\% \quad (2.2)$$

donde r_j es el j -ésimo bit de la respuesta y n es el ancho en bits de la PUF. Un valor cercano a 50 % indica que la PUF no presenta sesgo significativo hacia ceros o unos.

Confiabilidad (fiabilidad)

La confiabilidad cuantifica la reproducibilidad de las respuestas de una PUF ante el mismo desafío bajo las mismas condiciones operativas. Se evalúa midiendo la distancia de Hamming fraccional intra-dispositivo (*intra-HD*) entre la respuesta de referencia y respuestas

obtenidas en evaluaciones sucesivas [22]:

$$\text{Confiabilidad} = 100\% - \text{BER} \quad (2.3)$$

donde BER (*Bit Error Rate*) es la tasa de error de bit promedio sobre m evaluaciones:

$$\text{BER} = \frac{1}{m} \sum_{i=1}^m \frac{\text{HD}(R_{\text{ref}}, R_i)}{n} \times 100\% \quad (2.4)$$

con R_{ref} como la respuesta de referencia (obtenida por votación por mayoría), R_i como la i -ésima respuesta y $\text{HD}(\cdot, \cdot)$ la distancia de Hamming entre dos vectores binarios. Idealmente, la confiabilidad debe ser del 100 % (BER = 0); en la práctica, valores superiores al 95 % se consideran aceptables para aplicaciones de identificación [4, 22].

Unicidad

La unicidad mide la capacidad de una PUF para distinguir dispositivos individuales. Se evalúa mediante la distancia de Hamming fraccional inter-dispositivo (*inter-HD*) entre las respuestas de referencia de diferentes chips ante el mismo desafío [22]:

$$\text{Inter-HD} = \frac{2}{k(k-1)} \sum_{i=1}^{k-1} \sum_{j=i+1}^k \frac{\text{HD}(R_i, R_j)}{n} \times 100\% \quad (2.5)$$

donde k es el número de dispositivos evaluados. El valor ideal es 50 %, lo que indica máxima separación estadística entre los identificadores de dispositivos diferentes [22].

En la evaluación intra-dispositivo (cuando solo se dispone de un chip), la distancia de Hamming inter-desafío permite estimar indirectamente la variabilidad: se calculan las respuestas de referencia para m desafíos diferentes y se mide la HD promedio entre todos los pares de respuestas. Un valor cercano al 50 % indica que el PLPUF es capaz de generar respuestas suficientemente diversas ante desafíos distintos [4].

Alias de bit (*Bit-Alias*)

El alias de bit mide la tendencia de cada posición individual de la respuesta a ser predominantemente cero o uno a través de múltiples dispositivos [23]. Para la posición j , el alias de bit es la proporción de unos observados en esa posición sobre k dispositivos. El valor ideal es 0,5 para cada posición; desviaciones significativas indican bits con baja entropía que podrían ser explotados por un atacante [23]. Wilde y Pehl [23] demostraron la importancia de incorporar intervalos de confianza en la medición del alias de bit, señalando que se requiere un número elevado de muestras (del orden de cientos de dispositivos) para obtener estimaciones estadísticamente confiables.

Min-entropía

La min-entropía es una métrica más conservadora que la entropía de Shannon para cuantificar la impredecibilidad de una PUF, ya que considera la probabilidad de la respuesta más probable en lugar del promedio [31]. En sistemas que emplean algoritmos de datos auxiliares para la corrección de errores, la min-entropía condicional $\tilde{H}_\infty(X|Y)$ mide la impredecibilidad residual del secreto dado el conocimiento de los datos auxiliares públicos Y . Frisch et al. [31] propusieron un método basado en convolución de histogramas que permite estimar la min-entropía en escenarios con sesgos arbitrarios y correlaciones realistas, superando las limitaciones de los enfoques que asumen respuestas independientes e idénticamente distribuidas (IID).

Capítulo 3

Marco metodológico

Este capítulo describe la metodología empleada para el diseño, implementación, integración y evaluación del módulo PLPUF como periférico AXI dentro de un *System-on-Chip* basado en MicroBlaze V. Se presenta, en primer lugar, la estrategia metodológica adoptada y su justificación; a continuación, el diagrama de la secuencia del proceso seguido y la propuesta metodológica desglosada por objetivo específico; seguidamente, las herramientas de desarrollo utilizadas y el diagrama general de la solución; y, finalmente, las decisiones de diseño en cada etapa del proyecto (módulo HDL, interfaz AXI, integración SoC, software y protocolo de evaluación).

3.1 Estrategia metodológica

El desarrollo de sistemas digitales integrados que abarcan desde el diseño de hardware hasta el software embebido y la validación experimental requiere una estrategia metodológica que garantice la trazabilidad entre los entregables y la verificación progresiva de cada componente. Dentro de las aproximaciones disponibles en la ingeniería de sistemas, el modelo secuencial por fases, también conocido como modelo en cascada, se caracteriza por organizar el desarrollo en etapas lineales, donde cada fase produce artefactos verificables que constituyen la entrada directa de la fase siguiente [24].

A diferencia de los modelos iterativos o ágiles, que favorecen la entrega incremental y la retroalimentación temprana del usuario, el modelo secuencial resulta especialmente adecuado cuando los requisitos están bien definidos desde el inicio y existe una dependencia estricta entre los entregables de cada etapa. En el contexto de este proyecto, la relación de precedencia es clara: el módulo HDL del PLPUF (Fase 1) es prerrequisito de la interfaz AXI4-Lite (Fase 2); la interfaz empaquetada como IP es necesaria para la integración del SoC (Fase 3); y el sistema completo y funcional es indispensable para la evaluación estadística (Fase 4). Un modelo iterativo habría introducido riesgos de retrabajo sin beneficio significativo, dado que el alcance del proyecto estaba bien definido desde el anteproyecto y no se requería la participación de usuarios finales durante el desarrollo.

Por estas razones, se adoptó una **metodología secuencial por fases** organizada en cuatro etapas, cada una alineada con un objetivo específico del proyecto. La ejecución siguió el orden estricto Fase 1 → Fase 2 → Fase 3 → Fase 4, con puntos de verificación al final de cada fase para confirmar que los entregables satisfacían los criterios de aceptación antes de avanzar a la siguiente. La verificación en cada punto incluyó revisiones de simulación, pruebas de síntesis, pruebas de integración y análisis estadístico, según correspondiera a la naturaleza de los entregables.

3.2 Diagrama de la secuencia del proceso

La Figura 3.1 ilustra las cuatro fases del proyecto, sus relaciones de dependencia y los entregables asociados a cada fase. Cada fase se alinea con un objetivo específico (OE) y genera artefactos verificables que habilitan la fase subsiguiente.

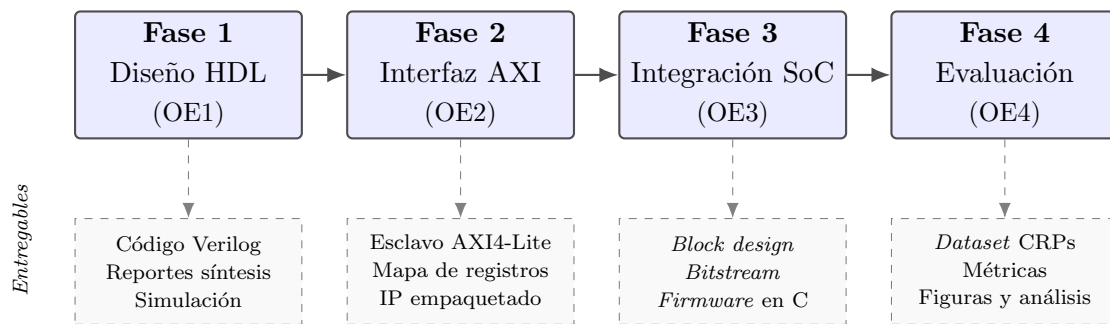


Figura 3.1: Metodología secuencial por fases del proyecto. Cada fase se alinea con un objetivo específico (OE) y produce entregables verificables que habilitan la fase siguiente.

Las cuatro fases se describen a continuación:

Fase 1 – Diseño del módulo PLPUF en HDL (OE1). Se implementó el núcleo PLPUF en Verilog siguiendo la arquitectura propuesta por Hori et al. [4]. Esta fase incluyó la definición de los parámetros del diseño, la implementación de la cadena de inversores y la máquina de estados finitos, la simulación funcional y la verificación de la síntesis con las restricciones necesarias para preservar el lazo combinacional.

Fase 2 – Desarrollo de la interfaz AXI4-Lite (OE2). A partir del módulo HDL validado en la fase anterior, se desarrolló la interfaz esclava AXI4-Lite que expone el PLPUF como periférico controlable por software. Se definió el mapa de registros, se implementó la lógica de lectura/escritura y se empaquetó el conjunto como un núcleo IP reutilizable.

Fase 3 – Integración del sistema SoC (OE3). Con el IP empaquetado, se construyó el sistema completo en Vivado IP Integrator: procesador MicroBlaze V, memoria BRAM,

UART, módulo de depuración y el periférico PLPUF. Se generó el *bitstream*, se exportó el diseño a Vitis y se desarrolló el *firmware* de control en lenguaje C, incluyendo el *driver* y las aplicaciones de prueba.

Fase 4 – Evaluación estadística (OE4). Finalmente, sobre el sistema funcional se ejecutó el protocolo de evaluación exhaustiva: 10 corridas independientes con 20 desafíos, 50 repeticiones y 10 duraciones de activación, acumulando más de 140 000 evaluaciones. Se calcularon las métricas de uniformidad, confiabilidad, distancia de Hamming y efecto avalancha, y se compararon los resultados con la literatura de referencia.

3.3 Propuesta metodológica por objetivo

La Tabla 3.1 resume la relación detallada entre cada objetivo específico, las actividades principales realizadas, los entregables esperados, las herramientas empleadas y las estrategias de verificación y validación aplicadas.

3.4 Herramientas de desarrollo

A continuación se describen las herramientas de hardware y software empleadas en el proyecto. La selección de cada herramienta se fundamenta en los requisitos del diseño y en la disponibilidad de recursos proporcionados por la Escuela de Ingeniería en Computadores del TEC.

3.4.1 Hardware: placa Basys 3

La plataforma de desarrollo seleccionada es la placa Basys3 de Digilent, descrita en la Sección 2.2.1. Su FPGA Artix-7 (XC7A35T-1CPG236C) ofrece recursos suficientes para instanciar el procesador MicroBlaze V junto con el periférico PLPUF y los módulos auxiliares (UART, controlador de memoria, gestión de reloj). El oscilador de 100 MHz integrado en la placa proporciona la frecuencia base del sistema, y el puerto USB-UART permite la comunicación serial con el computador anfitrión para la extracción de datos de prueba [3].

La placa fue proporcionada por la Escuela de Ingeniería en Computadores y constituye la única plataforma física de evaluación, lo que implica que los resultados experimentales corresponden a un único dispositivo FPGA [23].

3.4.2 AMD Vivado Design Suite

AMD Vivado Design Suite (versión 2025.1) se utilizó para todas las tareas de diseño de hardware: creación del *block design* con IP Integrator, empaquetado del IP personaliza-

Tabla 3.1: Propuesta metodológica por objetivo específico.

Objetivo específico	Actividades principales	Entregables	Herramientas	Verificación
OE1: Implementar el módulo PLPUF en HDL con ancho parametrizable.	Definir arquitectura PLPUF. Implementar en Verilog. Simulación y síntesis. Optimización de recursos.	Código HDL del PLPUF. Reportes de síntesis. Resultados de simulación.	Vivado. Verilog. <i>Testbenches</i> .	Simulación funcional. Revisión de tiempos y uso de recursos.
OE2: Desarrollar una interfaz AXI- <i>slave</i> para el control del módulo.	Diseño del mapa de registros. Implementación AXI- <i>slave</i> . Integración preliminar. Pruebas de lectura/escritura.	Módulo AXI- <i>slave</i> funcional. Mapa de registros documentado.	Vivado IP Integrator. Plantilla AXI4-Lite.	Validación con simulación AXI. Pruebas de comunicación con software.
OE3: Integrar el módulo en un sistema embebido completo con MicroBlaze V.	Construcción del SoC. Conexión del PLPUF vía AXI. Generación del <i>bitstream</i> . Desarrollo de software en Vitis.	Sistema SoC funcional. Software de control. <i>Bitstream</i> final.	Vivado. Vitis. UART. MicroBlaze V.	Comunicación serial exitosa. Depuración con Vitis.
OE4: Evaluar el PLPUF con métricas de seguridad.	Pruebas masivas de desafío-respuesta. Automatización de recolección. Análisis estadístico. Comparación con métricas esperadas.	<i>Dataset</i> de respuestas. Informe de evaluación. Gráficos y análisis.	<i>Scripts</i> en C. Python. Software de prueba.	Verificación repetida de muestras. Análisis estadístico. Revisión de consistencia.

do del PLPUF, síntesis lógica, implementación física (*place and route*), verificación de restricciones temporales y generación del *bitstream* [24]. Vivado también se empleó para la definición de restricciones de síntesis específicas del PLPUF (lazos combinacionales, atributos DONT_TOUCH) mediante archivos XDC [26].

3.4.3 AMD Vitis

AMD Vitis Unified Software Platform (versión 2025.1) se empleó para el desarrollo del software embebido. A partir del archivo `.xsa` exportado por Vivado, Vitis genera automáticamente la plataforma de hardware, el *Board Support Package* (BSP) y los controladores de periféricos [27]. Las aplicaciones de prueba se desarrollaron en lenguaje C, compiladas con la cadena de herramientas `riscv64-unknown-elf-gcc` incluida en Vitis. La carga del *bitstream* y del ejecutable en la FPGA se realizó a través de JTAG mediante el puerto USB J4 de la Basys 3 [27].

3.4.4 Terminal serial y Python

La comunicación entre la FPGA y el computador anfitrión se realizó mediante el periférico AXI UART Lite configurado a 9600 baudios. Para la captura de los datos de salida del sistema se utilizó un emulador de terminal serial (PuTTY). Los datos recolectados en formato texto fueron procesados posteriormente con *scripts* en Python para generar gráficos y cálculos estadísticos complementarios.

3.5 Diagrama general de la solución

La Figura 3.2 muestra el diagrama general del sistema, donde los componentes se agrupan por colores según su función: en **azul** los componentes del SoC (procesador, memoria e interconexión), en **verde** el periférico PLPUF desarrollado en este proyecto, en **naranja** los elementos de comunicación serial y en **rojo** el computador anfitrión externo. Los números circundados indican el flujo de operación descrito a continuación.

El sistema se compone de los siguientes grupos de componentes:

- **Componentes del SoC (azul)**: el procesador suave MicroBlaze V ejecuta el software de control; la memoria BRAM almacena instrucciones y datos; el AXI SmartConnect interconecta el procesador con los periféricos; y el módulo de depuración MDM V permite la carga de programas vía JTAG.
- **Periférico PLPUF (verde)**: el módulo desarrollado en este proyecto, conectado como esclavo AXI4-Lite al SmartConnect. Contiene la cadena de inversores generadora de entropía, la FSM de control y los registros de interfaz.

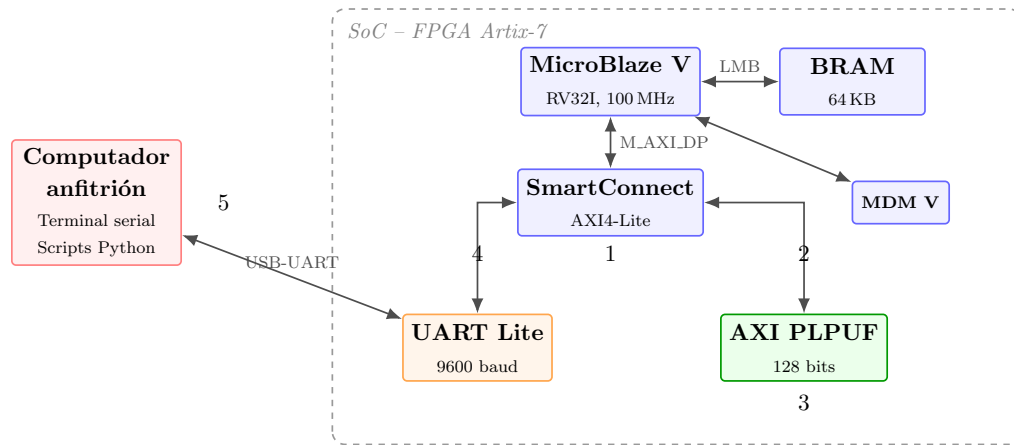


Figura 3.2: Diagrama general de la solución: sistema SoC con PLPUF integrado. Los colores agrupan los componentes por función y los números indican el flujo de operación.

- **Comunicación (naranja):** el periférico AXI UART Lite transmite los resultados al exterior a través del puerto USB-UART de la Basys 3.
- **Computador anfitrión (rojo):** recibe los datos por terminal serial y ejecuta los *scripts* de Python para el análisis estadístico.

El flujo de operación, indicado por los números circundados en la Figura 3.2, es el siguiente:

- 1 El software embebido, ejecutándose en MicroBlaze V, genera un desafío de 128 bits y una duración de activación c , y los envía al periférico PLPUF a través del bus AXI4-Lite vía el SmartConnect.
- 2 El SmartConnect enruta la transacción AXI hacia el periférico PLPUF, donde se escribe el desafío en los registros CHAL_0 a CHAL_3, la duración en DURATION y el bit de inicio en CTRL.
- 3 El núcleo `plpuf_core` ejecuta la secuencia FSM: LOAD (1 ciclo) → ACTIVE (c ciclos) → DONE (1 ciclo), capturando la respuesta de 128 bits en los registros RESP_0 a RESP_3.
- 4 El software lee la respuesta y la transmite al computador anfitrión a través de la UART.
- 5 En el computador anfitrión, los datos son capturados por un emulador de terminal serial y procesados con *scripts* de Python para generar las métricas de evaluación y los gráficos correspondientes.

Este flujo se repite de forma automatizada para todas las combinaciones de desafíos y duraciones definidas en el protocolo de evaluación (Sección 3.10), permitiendo la recolección sistemática de los datos necesarios para la caracterización completa del PLPUF.

Las secciones siguientes describen en detalle el diseño de cada componente del sistema, comenzando por el núcleo PLPUF, luego la interfaz AXI4-Lite, la integración del SoC, el software de control y, finalmente, el protocolo de evaluación.

3.6 Diseño del módulo PLPUF en HDL

Esta sección describe las decisiones de diseño adoptadas para la implementación del núcleo PLPUF en Verilog, correspondiente al archivo `plpuf_core.v`. El diseño sigue la arquitectura propuesta por Hori et al. [4] y descrita en la Sección 2.3.2.

3.6.1 Visión general del módulo

La Figura 3.3 presenta el diagrama de primer nivel del módulo `plpuf_core`, mostrando su interfaz externa como caja negra.

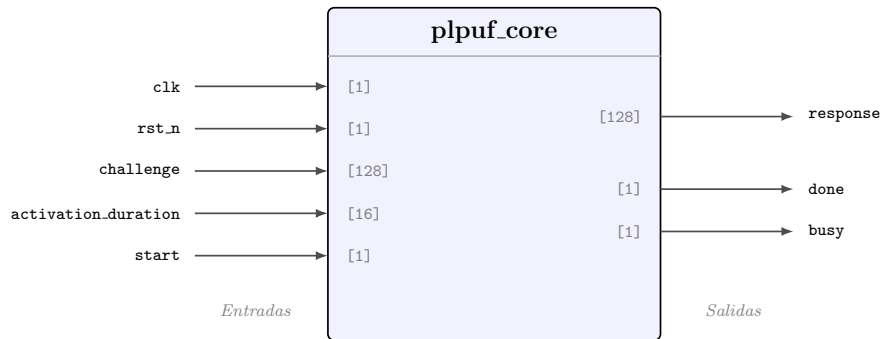


Figura 3.3: Diagrama de primer nivel del módulo `plpuf_core`: interfaz de puertos con anchos de señal.

La Tabla 3.2 describe en detalle cada puerto de la interfaz.

Tabla 3.2: Interfaz de puertos del módulo `plpuf_core`.

Puerto	Ancho	Dir.	Descripción
<code>clk</code>	1	E	Reloj del sistema.
<code>rst_n</code>	1	E	<i>Reset</i> activo en bajo.
<code>challenge</code>	128	E	Desafío de entrada.
<code>activation_duration</code>	16	E	Ciclos de oscilación libre (<i>c</i>).
<code>start</code>	1	E	Pulso de inicio de evaluación.
<code>response</code>	128	S	Respuesta generada por el PLPUF.
<code>done</code>	1	S	Pulso de un ciclo al completar.
<code>busy</code>	1	S	Evaluación en curso.

El protocolo de operación es sencillo: el maestro escribe el desafío de 128 bits y la duración de activación, emite un pulso en `start` y espera a que `done` se active. Al finalizar, la respuesta de 128 bits está disponible en el puerto `response`.

El diseño es parametrizable para facilitar su adaptación a diferentes anchos y polinomios primitivos. La Tabla 3.3 resume los cinco parámetros disponibles y sus valores por defecto.

Tabla 3.3: Parámetros configurables del módulo `plpuf_core`.

Parámetro	Valor	Descripción
WIDTH	128	Ancho en bits del PLPUF (tamaño del desafío y la respuesta).
DURATION_BITS	16	Ancho del contador de duración de activación, permite hasta $2^{16} - 1 = 65\,535$ ciclos.
TAP_1	125	Posición del primer <i>tap</i> XOR (corresponde al término x^{126} del polinomio).
TAP_2	101	Posición del segundo <i>tap</i> XOR (x^{102}).
TAP_3	98	Posición del tercer <i>tap</i> XOR (x^{99}).

Las posiciones de *tap* corresponden al polinomio primitivo $p(x) = x^{128} + x^{126} + x^{102} + x^{99} + 1$, el mismo utilizado en la propuesta original [4].

Internamente, el módulo se compone de dos bloques funcionales:

- **Cadena de inversores con retroalimentación XOR:** constituye el elemento generador de entropía, aprovechando las variaciones físicas del silicio (Sección 3.6.2).
- **Máquina de estados finitos (FSM) de 4 estados:** controla las fases de carga del desafío, oscilación libre y captura de la respuesta (Sección 3.6.3).

3.6.2 Cadena de inversores y retroalimentación

La cadena de inversores constituye el núcleo generador de entropía del PLPUF. Se compone de 128 celdas idénticas, generadas mediante un bloque `generate` de Verilog. Cada celda contiene un multiplexor y un inversor: durante la fase de carga, el multiplexor selecciona el bit correspondiente del desafío; durante la fase activa, selecciona la señal de retroalimentación del vecino, cerrando así el lazo combinacional.

La retroalimentación sigue la topología Fibonacci: la mayoría de las celdas reciben la salida de la celda adyacente, mientras que las celdas en las posiciones de *tap* (125, 101 y 98) reciben una XOR entre la salida de su vecina y la de la celda 0, reproduciendo el polinomio primitivo de 128 bits. Esta estructura forma un lazo puramente combinacional (sin registros de desplazamiento) que oscila libremente durante la fase activa, generando la entropía a partir de las variaciones de retardo del silicio [4].

3.6.3 Máquina de estados finitos

El control del PLPUF se implementó mediante una FSM de cuatro estados codificada en 2 bits, cuyo diagrama de transiciones se muestra en la Figura 2.7. La secuencia de operación es la siguiente:

1. **S_IDLE:** estado de reposo. Al recibir el pulso **start**, la FSM almacena el desafío y la duración de activación, y transiciona a la fase de carga.
2. **S_LOAD:** el desafío se propaga a través de la cadena de inversores durante un ciclo de reloj, estableciendo el estado inicial del lazo.
3. **S_ACTIVE:** la retroalimentación se cierra y la cadena oscila libremente durante c ciclos. Al finalizar, el estado de la cadena se captura como respuesta.
4. **S_DONE:** se emite el pulso **done** y la FSM retorna a reposo.

3.6.4 Restricciones de síntesis

El lazo combinacional del PLPUF genera violaciones DRC (LUTLP-1) en Vivado, ya que la herramienta identifica correctamente la existencia de caminos combinacionales cíclicos. Para permitir la síntesis y la implementación sin errores, se definieron las siguientes restricciones en el archivo `plpuf.xdc`:

1. **ALLOW_COMBINATORIAL_LOOPS:** aplicado a las redes del lazo (`*plpuf_core_inst/core_in*`) para permitir que Vivado acepte el ciclo combinacional.
2. **DONT_TOUCH:** aplicado a todas las celdas del núcleo (`*plpuf_core_inst*`) para prevenir la optimización o eliminación de inversores y multiplexores durante la síntesis.
3. **set_false_path:** aplicado a las mismas redes del lazo para indicar al motor de temporización que no analice estos caminos, ya que no representan rutas de datos síncronas convencionales.

Los *wildcards* utilizan nombres de redes post-síntesis (`core_in[N]`, `core_in_inferred_i_N`) y no los nombres del bloque **generate** (`gen_core`), ya que estos últimos no se preservan tras la síntesis.

3.7 Diseño de la interfaz AXI4-Lite

Para integrar el PLPUF como periférico controlable por software, se implementó una interfaz AXI4-Lite esclava siguiendo la plantilla estándar generada por Vivado [26]. Esta

interfaz permite al procesador MicroBlaze V configurar el desafío y la duración de activación, iniciar la evaluación y leer la respuesta mediante operaciones de lectura/escritura sobre registros mapeados en memoria [2].

La Figura 3.4 presenta el diagrama del diseño de la interfaz, mostrando la relación entre los canales AXI, el bloque de registros y el núcleo PLPUF.

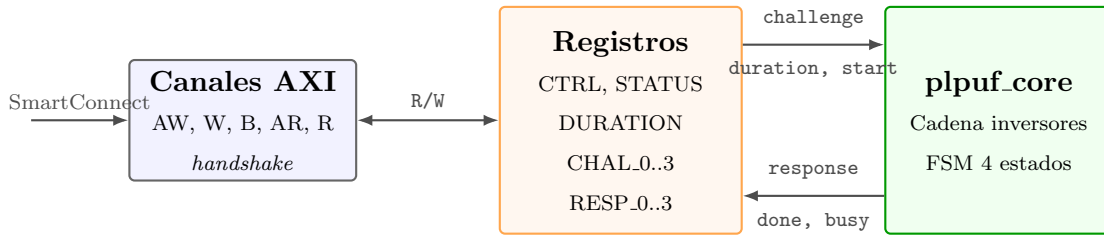


Figura 3.4: Diagrama del diseño de la interfaz AXI4-Lite: los canales AXI comunican el bus con el bloque de registros, que a su vez controla el núcleo PLPUF.

3.7.1 Mapa de registros

El periférico expone 11 registros activos dentro de un espacio de direcciones de 64 bytes (`C_S_AXI_ADDR_WIDTH = 6`, equivalente a 16 ranuras de 32 bits). La Tabla 3.4 describe el mapa de registros completo.

Tabla 3.4: Mapa de registros del periférico AXI PLPUF.

Offset	Nombre	R/W	Descripción
0x00	CTRL	R/W	[0] start : inicia evaluación (auto-limpia). [1] soft_reset : reinicio del núcleo (auto-limpia).
0x04	STATUS	R	[0] busy : evaluación en curso. [1] done : evaluación completada (persistente).
0x08	DURATION	R/W	[15:0] Duración de activación en ciclos de reloj.
0x0C	CHAL_0	R/W	Bits [31:0] del desafío.
0x10	CHAL_1	R/W	Bits [63:32] del desafío.
0x14	CHAL_2	R/W	Bits [95:64] del desafío.
0x18	CHAL_3	R/W	Bits [127:96] del desafío.
0x1C	RESP_0	R	Bits [31:0] de la respuesta.
0x20	RESP_1	R	Bits [63:32] de la respuesta.
0x24	RESP_2	R	Bits [95:64] de la respuesta.
0x28	RESP_3	R	Bits [127:96] de la respuesta.

3.7.2 Lógica del esclavo AXI

El módulo `axi_plpuf_slave_lite_v1_0_S00_AXI` implementa la lógica completa del esclavo AXI4-Lite y la instanciación del `plpuf_core`. Las características del diseño son:

- **Máquinas de estado AXI:** se mantuvieron las FSM de la plantilla de Vivado para los canales de escritura (estados `Idle`, `Waddr`, `Wdata`) y lectura (`Idle`, `Raddr`, `Rdata`), garantizando conformidad con el protocolo *handshake* VALID/READY [2].
- **Escritura con tres niveles de prioridad:** en un único bloque `always` con asignaciones no bloqueantes, se definen tres capas de escritura donde la última asignación prevalece:
 1. Auto-limpieza de los bits `CTRL[1:0]` (prioridad baja).
 2. Escrituras del maestro AXI con *byte strobes* (`WSTRB`) (prioridad media).
 3. Actualización de registros `STATUS` y `RESP` por el hardware del PLPUF (prioridad alta).
- **Bandera done persistente (*sticky*):** el bit `done` en `STATUS` se activa con el pulso `plpuf_done` y permanece activo hasta que el software escribe un nuevo `start` o un `soft_reset`.
- **Registros de solo lectura:** los registros `RESP_0` a `RESP_3` se actualizan exclusivamente desde el hardware; cualquier escritura AXI a estas direcciones es sobrescrita en el mismo ciclo por la capa de prioridad alta, garantizando la integridad de la respuesta.

La interconexión entre los registros y el núcleo PLPUF se realiza de la siguiente manera: el desafío de 128 bits se forma concatenando los registros `CHAL_3` a `CHAL_0` (`{slv_reg6, slv_reg5, slv_reg4, slv_reg3}`); la duración de activación se toma de los 16 bits inferiores de `DURATION`; y la señal de *reset* del núcleo combina el *reset* global del bus AXI con el bit de `soft_reset` del registro `CTRL`.

3.7.3 Empaquetado del IP

Para que el módulo PLPUF sea reutilizable dentro de cualquier *block design* de Vivado, se empaquetó como un IP (*Intellectual Property*) mediante el flujo de IP Packager de Vivado [26]. El proceso genera un descriptor IP-XACT (`component.xml`) que encapsula:

- Los tres archivos HDL: `axi_plpuf.v`, `axi_plpuf_slave_lite_v1_0_S00_AXI.v` (esclavo AXI + núcleo) y `plpuf_core.v`.
- El *driver* en C (`axi_plpuf.h`, `axi_plpuf.c`) para generación automática del *Board Support Package* (BSP) en Vitis.

- Los parámetros de la interfaz de reloj (`A_BUSIF = S00_AXI`, `A_RESET = s00_axi_aresetn`, `FREQ_HZ = 100000000`), necesarios para que el SmartConnect verifique la compatibilidad de dominios de reloj.

La jerarquía de módulos del IP se compone de tres niveles, como se ilustra en la Figura 3.5.

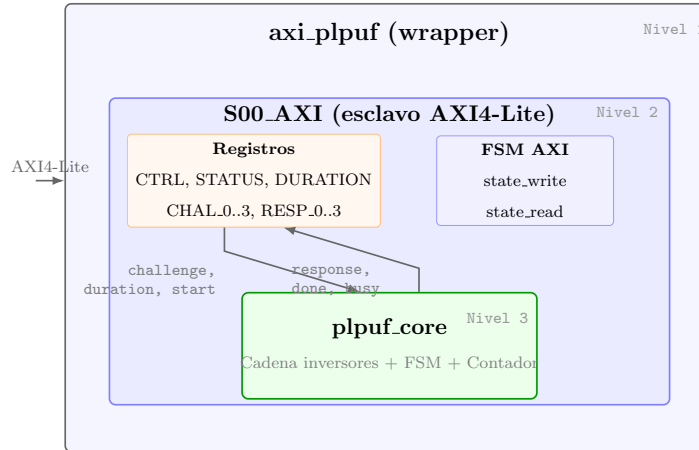


Figura 3.5: Jerarquía de módulos del IP AXI PLPUF: tres niveles de encapsulamiento con el *wrapper*, el esclavo AXI4-Lite (registros y FSM AXI) y el núcleo PLPUF.

El *wrapper* `axi_plpuf` expone la interfaz AXI4-Lite al exterior. El módulo `S00_AXI` contiene la lógica de decodificación de direcciones, los *handshakes* de los cinco canales AXI, los 16 registros de 32 bits (de los cuales 11 están activos) y las máquinas de estado de escritura y lectura. En el nivel inferior, el módulo `plpuf_core` recibe del esclavo AXI las señales de desafío, duración, inicio y *reset*, y retorna la respuesta junto con las banderas de estado.

3.8 Integración del sistema SoC

Esta sección describe la construcción del sistema completo dentro de Vivado IP Integrator, el flujo de síntesis e implementación, y la exportación del diseño hacia Vitis para el desarrollo de software.

3.8.1 Diseño de bloques en Vivado

El sistema SoC se construyó como un *block design* en Vivado IP Integrator [24]. La Figura 3.6 muestra el diagrama de bloques completo del sistema tal como aparece en el entorno gráfico de IP Integrator, donde cada rectángulo representa un núcleo IP con sus puertos de entrada y salida, y las líneas de interconexión representan los buses AXI, las señales de reloj, *reset* y depuración. Los núcleos IP instanciados y sus funciones dentro del sistema son los siguientes:

- **Clocking Wizard (clk_wiz_0):** genera la señal de reloj de 100 MHz a partir del oscilador de la placa (pin W5).
- **MicroBlaze V (microblaze_riscv_0):** procesador suave RISC-V configurado con la ISA base RV32I, *pipeline* de 5 etapas, depuración habilitada e interfaces AXI de datos (M_AXI_DP) [1].
- **Memoria local (BRAM + LMB):** bloque de memoria RAM, 32 KB para instrucciones y 32 KB para datos, conectado al procesador mediante el bus local de memoria (LMB) [25].
- **MicroBlaze Debug Module V (mdm_1):** módulo de depuración que permite la conexión JTAG para carga de programas y depuración paso a paso [1].
- **Processor System Reset (rst_clk_wiz_0_100M):** genera las señales de *reset* sincronizadas para el procesador y los periféricos.
- **AXI SmartConnect (axi_smc):** interconexión AXI con un puerto maestro y dos puertos esclavos: uno para el UART y otro para el PLPUF. Realiza la traducción de protocolo y el arbitraje de direcciones [26].
- **AXI UART Lite (axi_uartlite_0):** periférico UART conectado al puerto USB-UART de la Basys 3, configurado a 9 600 baudios para la comunicación serial con el computador anfitrión.
- **AXI PLPUF (axi_plpuf_0):** el periférico PLPUF desarrollado en este proyecto, conectado al segundo puerto esclavo del SmartConnect.

La asignación de direcciones del sistema se resume en la Tabla 3.5.

Tabla 3.5: Mapa de direcciones del sistema SoC.

Periférico	Dirección base	Rango
BRAM local (datos e instrucciones)	0x00000000	32 KB
AXI UART Lite	0x40600000	64 KB
AXI PLPUF	0x44A00000	64 KB

3.8.2 Flujo de síntesis e implementación

El flujo de construcción del *bitstream* siguió los pasos estándar de Vivado [24]:

1. **Generación de productos de salida:** se generaron los productos de salida de todos los IPs del *block design*, incluyendo los *wrappers* HDL.



Figura 3.6: Diseño de bloques del sistema SoC en Vivado IP Integrator, mostrando la interconexión entre el procesador MicroBlaze V, la memoria local (BRAM/LMB), el módulo de depuración (MDM V), el AXI SmartConnect, el periférico UART Lite y el periférico PLPUF desarrollado en este proyecto.

2. **Síntesis lógica:** Vivado sintetiza el diseño completo. En esta etapa se producen las advertencias DRC LUTLP-1 por el lazo combinacional del PLPUF, las cuales son resueltas por las restricciones XDC descritas en la Sección 3.6.4.
3. **Implementación:** Vivado realiza el *placement* y *routing* del diseño sobre los recursos de la FPGA Artix-7. Las restricciones DONT_TOUCH aseguran que las celdas del PLPUF no sean reubicadas ni optimizadas.
4. **Verificación de temporización:** se verificó que todas las rutas de datos síncronas cumplan las restricciones temporales. Las rutas del lazo combinacional están excluidas del análisis mediante `set_false_path`.
5. **Generación del *bitstream*:** se habilitó la compresión del *bitstream* para reducir el tiempo de carga.

3.8.3 Exportación a Vitis

Una vez generado el *bitstream*, se exportó el hardware desde Vivado en formato `.xsa` (*Xilinx Support Archive*), que incluye:

- La descripción del hardware (mapa de direcciones, IPs instanciados, parámetros).
- El *bitstream* de la FPGA.
- Los metadatos necesarios para que Vitis genere el BSP con los controladores de los periféricos [27].

En Vitis, se creó una plataforma de hardware a partir del archivo `.xsa` y se configuró el procesador MicroBlaze V como objetivo de compilación. El BSP generado automáticamente incluye el *driver* del periférico PLPUF (`axi_plpuf.h` y `axi_plpuf.c`),

gracias a que estos archivos fueron incluidos en el paquete IP durante el proceso de empaquetado descrito en la Sección 3.7.3 [27].

3.9 Desarrollo del software de control

El software embebido se organizó en tres componentes: un *driver* reutilizable para el periférico PLPUF, una aplicación de validación básica y una aplicación de evaluación exhaustiva. La Figura 3.7 muestra la arquitectura en capas del software.

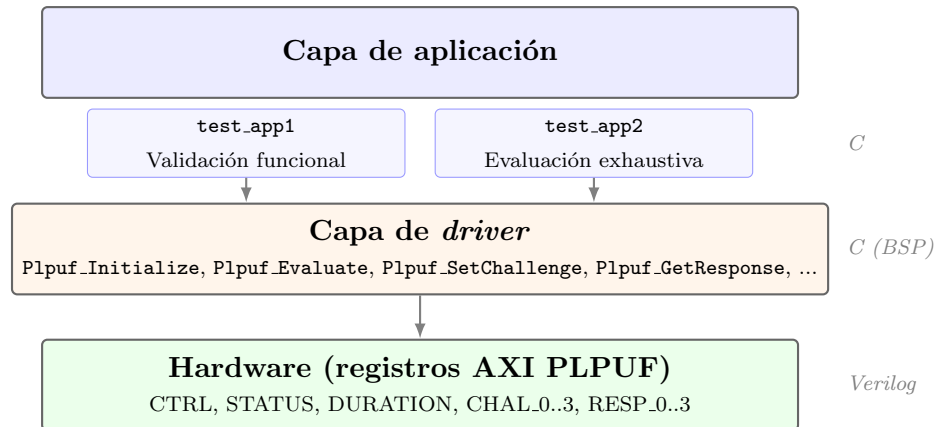


Figura 3.7: Arquitectura en capas del software de control: las aplicaciones de prueba utilizan el *driver* del PLPUF, que a su vez accede a los registros del periférico mediante operaciones de lectura/escritura AXI.

3.9.1 Driver en C para el PLPUF

Se desarrolló un *driver* en C siguiendo las convenciones del ecosistema AMD/Xilinx (macros `Xil_In32/Xil_Out32`, aserciones `Xil_Assert`). El *driver* expone una API estructurada en tres niveles:

1. **Inicialización y *reset*:** `Plpuf_Initialize()` configura la dirección base del periférico, ejecuta un *soft reset* y establece la duración de activación por defecto ($c = 5$). `Plpuf_SoftReset()` reinicia el núcleo PLPUF mediante el bit 1 del registro CTRL.
2. **Configuración y control:** `Plpuf_SetDuration()` escribe la duración de activación; `Plpuf_SetChallenge()` escribe los 128 bits del desafío en los cuatro registros CHAL; `Plpuf_Start()` inicia la evaluación escribiendo el bit 0 del registro CTRL.
3. **Lectura de resultados:** `Plpuf_IsBusy()` y `Plpuf_IsDone()` consultan el registro STATUS; `Plpuf_WaitDone()` realiza un ciclo de espera activa hasta que la evaluación finalice; `Plpuf_GetResponse()` lee los cuatro registros RESP y almacena la respuesta de 128 bits.

Adicionalmente, la función de conveniencia `Plpuf_Evaluate()` encapsula el ciclo completo de evaluación en una sola llamada. La Figura 3.8 muestra el diagrama de flujo de esta función.

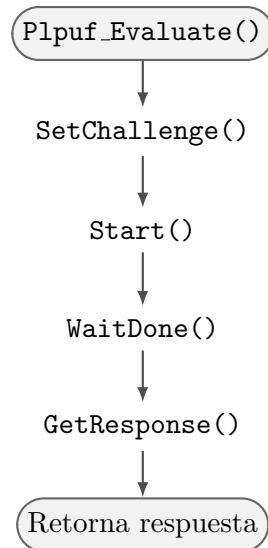


Figura 3.8: Diagrama de flujo de la función `Plpuf_Evaluate()`: ciclo completo de evaluación desafío-respuesta.

3.9.2 Aplicación de validación básica (`test_app1`)

La primera aplicación de prueba (`plpuf_test_app1`) se diseñó como una validación funcional rápida del sistema, verificando que todos los componentes (procesador, bus AXI, periférico PLPUF, UART) operan correctamente. Las pruebas realizadas son:

1. **Lectura/escritura de registros:** se escribe un valor conocido en `DURATION` y `CHAL` y se verifica que la lectura retorne el mismo valor.
2. **Evaluación individual:** se ejecuta una evaluación con un desafío conocido y se verifica que la respuesta sea no nula (128 bits $\neq 0$).
3. **Repetibilidad:** se ejecuta el mismo desafío 5 veces consecutivas y se comparan las respuestas. Se espera que sean idénticas o muy similares (dependiendo de la duración de activación).
4. **Unicidad inter-desafío:** se evalúan dos desafíos diferentes y se verifica que las respuestas sean distintas.
5. **Barrido de duración:** se evalúa un mismo desafío con duraciones $c = 1, \dots, 10$ y se imprimen todas las respuestas por UART para verificar que varían con c .

3.9.3 Aplicación de evaluación exhaustiva (test_app2)

La segunda aplicación (`plpuf_test_app2`) implementa el protocolo completo de evaluación estadística del PLPUF, ejecutando aproximadamente 14 000 evaluaciones por corrida sobre el hardware. Los parámetros de configuración son:

- 20 desafíos aleatorios (generados por un PRNG `xorshift32` con semilla `0xDEADBEEF` para reproducibilidad).
- 50 repeticiones por par desafío-duración (para votación por mayoría).
- 10 duraciones de activación ($c = 1, \dots, 10$).
- 32 posiciones de bit para pruebas de efecto avalancha.
- 9 evaluaciones con votación por mayoría para las respuestas *golden* del efecto avalancha.

La aplicación calcula las siguientes métricas directamente en el dispositivo utilizando aritmética de punto fijo (multiplicación por 100 o 10 000 y división entera para obtener dos decimales):

- **Uniformidad** por desafío y promedio por duración.
- **Confiabilidad** ($100\% - \text{BER}$) respecto a una respuesta de referencia obtenida por votación por mayoría (`golden_from_counts`).
- **BER** (*Bit Error Rate*) promedio.
- **Distancia de Hamming intra-dispositivo** (entre repeticiones del mismo desafío).
- **Distancia de Hamming inter-desafío** (entre respuestas *golden* de desafíos diferentes con la misma duración).
- **Mapa de estabilidad de bits**: para cada posición de bit, se califica de 0 (siempre inestable) a 9 (máxima estabilidad), basado en la fracción de repeticiones que coinciden con la respuesta *golden*.
- **Efecto avalancha**: se aplican cambios de un solo bit en el desafío y se mide la distancia de Hamming respecto a la respuesta *golden* original [22].

Todos los resultados se imprimen por UART en formato legible para su captura y procesamiento posterior.

3.10 Protocolo de evaluación del PLPUF

El protocolo de evaluación sigue las recomendaciones de Maiti et al. [22] y los criterios experimentales empleados por Hori et al. [4] en la evaluación original del PLPUF. La evaluación se realizó sobre un único dispositivo FPGA (Basys 3), por lo que las métricas reportadas corresponden a una evaluación intra-dispositivo [23].

3.10.1 Recolección de pares desafío-respuesta

La recolección de datos se estructuró de la siguiente manera:

1. Para cada duración de activación $c \in \{1, 2, \dots, 10\}$:
 - 1.1. Se generan 20 desafíos pseudoaleatorios de 128 bits mediante el generador `xorshift32` con semilla fija.
 - 1.2. Para cada desafío, se ejecutan 50 evaluaciones consecutivas, obteniendo 50 respuestas de 128 bits.
 - 1.3. Se construye la respuesta de referencia (*golden*) mediante votación por mayoría bit a bit: para cada posición, el valor del bit *golden* es el que aparece en más de 25 de las 50 repeticiones.
2. Total de evaluaciones: $10 \times 20 \times 50 = 10\,000$ evaluaciones principales, más las evaluaciones adicionales para el efecto avalancha.

3.10.2 Cálculo de métricas

A partir de los datos recolectados, se calculan las métricas definidas en la Sección 2.4.2 de la siguiente manera:

- **Uniformidad:** para cada respuesta *golden*, se calcula el peso de Hamming (*Hamming weight*) y se divide entre 128 para obtener la proporción de unos. El valor ideal es 50 % [22].
- **Confiabilidad y BER:** para cada par desafío-duración, se calcula la distancia de Hamming fraccional entre cada una de las 50 respuestas y la respuesta *golden*, obteniendo el BER promedio. La confiabilidad se define como $100\% - \text{BER}$ [22].
- **Distancia de Hamming intra-dispositivo:** promedio de las distancias de Hamming entre todas las repeticiones y la respuesta *golden* para un mismo desafío. El valor ideal es 0 [4].

- **Distancia de Hamming inter-desafío:** al disponer de un solo dispositivo, la métrica de unicidad inter-dispositivo no puede calcularse directamente. Como aproximación, se calcula la distancia de Hamming fraccional promedio entre las respuestas *golden* de los 20 desafíos diferentes para una misma duración. Un valor cercano a 50 % indica que el PLPUF genera respuestas suficientemente diversas ante desafíos distintos [4].

3.10.3 Efecto avalancha

El efecto avalancha mide la sensibilidad de la PUF a cambios mínimos en el desafío: al modificar un solo bit del desafío, idealmente el 50 % de los bits de la respuesta debería cambiar [22]. El protocolo implementado es:

1. Se selecciona un desafío base y se obtiene su respuesta *golden* con votación por mayoría (9 repeticiones).
2. Para cada una de las primeras 32 posiciones de bit del desafío:
 - 2.1. Se genera un desafío modificado invirtiendo únicamente el bit en la posición j .
 - 2.2. Se obtiene la respuesta *golden* del desafío modificado (9 repeticiones con votación por mayoría).
 - 2.3. Se calcula la distancia de Hamming fraccional entre la respuesta original y la modificada.
3. Se reporta el promedio de las 32 distancias como el efecto avalancha del PLPUF.

Cabe señalar que el PLPUF, al basarse en una estructura LFSR con difusión limitada, exhibe típicamente un efecto avalancha inferior al 50 % ideal. Alsharkawy et al. [9] documentaron esta limitación y su relación con la duración de activación.

Capítulo 4

Descripción del trabajo realizado

4.1 Proceso para llegar a la solución aplicada

Esta sección detalla el proceso metodológico de ingeniería seguido para diseñar e implementar la solución, abordando secuencialmente cada uno de los objetivos específicos definidos para el proyecto. Se describen las herramientas de análisis empleadas, las abstracciones realizadas y las decisiones de diseño fundamentales que permitieron construir la plataforma de evaluación.

4.1.1 OE1: Implementación del módulo PLPUF en HDL

El proceso de implementación del núcleo generador de entropía inició con la abstracción del circuito PLPUF propuesto por Hori et al. [4] hacia un modelo de descripción de hardware puramente combinacional. Se decidió emplear el lenguaje Verilog, optando por una arquitectura parametrizable que facilitara el ajuste del ancho del desafío (configurado a 128 bits) y las posiciones de las compuertas lógicas de retroalimentación (polinomio primitivo).

Una decisión de diseño crítica en esta etapa fue cómo preservar el comportamiento oscilatorio del circuito durante la síntesis. Las herramientas de síntesis (EDA), como AMD Vivado, están diseñadas para optimizar la lógica combinacional y eliminar bucles asíncronos que consideran como errores de diseño. Para evitar que el sintetizador eliminara los inversores, que constituyen la fuente de entropía física del sistema, se aplicaron restricciones de diseño explícitas. Se utilizó el atributo `DONT_TOUCH` sobre las celdas del núcleo para prevenir la absorción de lógica, y la restricción `ALLOW_COMBINATORIAL_LOOPS` en el archivo XDC para instruir a la herramienta que permitiera la existencia del lazo cerrado. Estas decisiones de síntesis fueron fundamentales para garantizar que el comportamiento analógico del silicio (los retardos de propagación microscópicos de cada compuerta debidos a las variaciones del proceso de fabricación) se preservara y pudiera ser explotado en el entorno digital del FPGA.

4.1.2 OE2: Desarrollo de la interfaz AXI-slave

Para habilitar el control del módulo PLPUF desde un procesador en un sistema embebido, se aplicaron principios de diseño de interfaces estandarizadas, seleccionando el protocolo AMBA AXI4-Lite por su ligereza y amplia adopción. El diseño de la interfaz partió de la definición de un mapa de registros *memory-mapped I/O* (MMIO) estructurado. Se abstra-jo el control del hardware en 11 registros activos de 32 bits, organizados lógicamente para separar funciones de configuración (desafío, duración de activación), control operativo (inicio, reinicio) y estado (bancos de respuesta, banderas).

Una técnica de diseño clave aplicada durante el desarrollo de esta interfaz fue la implementación de una lógica de prioridades de escritura en tres niveles. Este enfoque garantizó que el hardware interno del PLPUF (prioridad alta) siempre pudiera actualizar los registros de respuesta de forma segura y oportuna, evitando condiciones de carrera o sobreescritura frente a las peticiones concurrentes del procesador (prioridad baja/media). Esta abstracción aseguró que los registros de respuesta se comportaran estrictamente como de «solo lectura» desde la perspectiva del software. Finalmente, el código HDL integrado se sintetizó y se empaquetó como un núcleo de propiedad intelectual (IP) reutilizable utilizando Vivado IP Packager, estandarizando su instanciación para ser compatible con el flujo de diseño gráfico de sistemas.

4.1.3 OE3: Integración del sistema SoC

La síntesis del sistema completo se realizó empleando la herramienta Vivado IP Integrator, aplicando el principio de diseño modular basado en componentes. Se construyó el *System-on-Chip* interconectando el módulo IP del PLPUF (creado en la etapa anterior) con componentes de biblioteca estándar: el procesador suave MicroBlaze V (configurado con la arquitectura abierta RV32I para eficiencia de área), bloques de memoria local BRAM de 64 KB para almacenamiento de instrucciones y datos, un controlador UART para comunicación exterior, y el bloque AXI SmartConnect encargado del arbitraje y enrutamiento de transacciones en el bus del sistema.

A nivel de *software*, el proceso de integración involucró el desarrollo de una arquitectura en capas utilizando la plataforma AMD Vitis. Se programó un controlador (*driver*) de bajo nivel en lenguaje C para abstraer las direcciones físicas del hardware en una API de funciones intuitivas (como `Plpuf_SetChallenge` o `Plpuf_Start`). Sobre esta capa de abstracción, se construyeron las rutinas de *firmware* para la validación funcional y la evaluación exhaustiva. La decisión estratégica de trasladar la lógica de recolección de datos masiva y los cálculos estadísticos preliminares, como el cómputo de respuestas de referencia (*golden*) por votación mayoritaria, directamente al microprocesador del SoC, permitió optimizar el tiempo de ejecución del experimento y reducir sustancialmente el volumen de datos transferidos hacia el computador anfitrión.

4.1.4 OE4: Protocolo de evaluación estadística

El proceso de análisis para validar el desempeño de seguridad del módulo requirió la definición y aplicación de un protocolo experimental riguroso, fundamentado en el marco sistemático propuesto por Maiti et al. [22]. Se diseñó un experimento automatizado que ejecutó 10 corridas independientes sobre el sistema físico. Para estructurar adecuadamente el espacio de prueba, se estableció para cada corrida un barrido sobre 10 duraciones de activación diferentes ($c = 1, \dots, 10$), evaluando en cada paso 20 desafíos pseudoaleatorios fijos, a razón de 50 repeticiones por cada par desafío-duración.

Este diseño metodológico garantizó la representatividad y significancia estadística de los datos, acumulando aproximadamente 140 000 evaluaciones del PLPUF bajo condiciones operativas controladas. El cálculo final de las métricas (uniformidad, confiabilidad, distancia de Hamming y efecto avalancha) se resolvió mediante una estrategia híbrida: el preprocesamiento *in-situ* en el FPGA y el postprocesamiento en el computador anfitrión mediante *scripts* de Python, aprovechando librerías de visualización de datos para abstraer las cifras tabulares en representaciones gráficas de análisis.

4.2 Análisis de los resultados finales

En esta sección se evidencia el análisis cuantitativo y cualitativo de los resultados generados por el producto desarrollado. La presentación se divide en correspondencia con la validación de cada objetivo específico, proporcionando tablas y figuras que sustentan el grado de cumplimiento técnico y las implicaciones de la plataforma.

4.2.1 Resultados de la implementación HDL (OE1)

La validación del diseño HDL se centró en corroborar la viabilidad de implementación en dispositivos con recursos limitados y en demostrar que el circuito efectivamente genera respuestas no predecibles fundamentadas en la entropía física del silicio.

La Tabla 4.1 muestra la utilización de recursos extraída del reporte de síntesis de Vivado tras la fase de *place and route*. El sistema SoC completo ocupa apenas el 12.29 % de las LUTs y el 6.37 % de los *flip-flops* disponibles en la FPGA Artix-7 XC7A35T, confirmando que la solución es ampliamente implementable incluso en dispositivos modestos.

Para evaluar la eficiencia del módulo PLPUF de forma aislada, la Tabla 4.2 reporta los recursos consumidos exclusivamente por el periférico `axi_plpuf_0`. Requiere únicamente 484 LUTs (2.33 %) y 651 *flip-flops* (1.56 %). De estos, 128 LUTs corresponden a los inversores del núcleo combinacional preservados con el atributo `DONT_TOUCH`. Cabe destacar que el módulo no requiere bloques DSP ni memoria BRAM, dejando estos recursos disponibles para funciones de aceleración criptográfica en futuras aplicaciones.

De forma más trascendente, el éxito del OE1 se demuestra mediante el comportamiento

Tabla 4.1: Utilización de recursos del sistema SoC completo sobre la FPGA Artix-7 XC7A35T tras la implementación física.

Recurso	Usados	Disponibles	Util. (%)
<i>Slice</i> LUTs	2 556	20 800	12.29
LUT como lógica	2 449	20 800	11.77
LUT como memoria	107	9 600	1.11
<i>Slice</i> Registers	2 650	41 600	6.37
<i>Slices</i> ocupados	978	8 150	12.00
F7 Muxes	83	16 300	0.51
F8 Muxes	33	8 150	0.40
Bloques RAM (RAMB36E1)	16	50	32.00

Tabla 4.2: Utilización de recursos del periférico AXI PLPUF y del procesador MicroBlaze V, según el reporte de síntesis individual.

Componente	LUTs	FFs	F7/F8 Muxes	BRAM
AXI PLPUF (<code>axi_plpuf_0</code>)	484	651	64 / 32	0
MicroBlaze V (<code>microblaze_riscv_0</code>)	1 768	1 285	19 / 1	0

estadístico de la uniformidad de bits en la salida (analizada en detalle en la Sección 4.2.4). Dado que el circuito diseñado es estricta y puramente lógico digital estándar, la única forma de que sus salidas se asemejen a distribuciones pseudoaleatorias con proporciones de ceros y unos balanceadas ($\sim 50\%$) es debido a que la lógica está explotando con éxito las minúsculas **variaciones físicas de los retardos** en los caminos de los transistores. La ausencia de sesgo en las mediciones empíricas certifica que las restricciones impuestas en Vivado funcionaron y el PLPUF extrae la entropía inherente del hardware, en lugar de optimizarse hacia un estado constante.

4.2.2 Pruebas de la interfaz AXI (OE2)

Antes de autorizar la evaluación masiva de seguridad, se realizaron pruebas de validación funcional específicas (`test_app1`) ejecutadas por el MicroBlaze V para comprobar rigurosamente el protocolo de comunicación de la interfaz AXI-slave.

Las pruebas ejecutadas y aprobadas en hardware incluyeron:

1. **Lectura y escritura de registros de configuración:** Se verificó que el procesador pudiera escribir y recuperar sin alteraciones los valores en los registros de DURATION y CHAL.
2. **Protección de lectura sobre registros de estado:** Se intentaron ejecutar escrituras indebidas sobre los registros de RESP (respuesta). La prueba comprobó

que la interfaz AXI descartó estos datos, garantizando la condición de *read-only* del núcleo generador y su inmunidad ante modificaciones por software.

3. **Soft reset e indicadores de estado:** Se constató que tras ejecutar una evaluación (donde la bandera `done` cambia a 1), la emisión de un comando de *soft reset* en el registro CTRL condujo a una limpieza exitosa de las banderas y la correcta restauración de la FSM al estado de reposo, indicando disponibilidad.

Estos resultados con diagnósticos **ALL PASS** en las 10 corridas certifican que el diseño de prioridad concurrente del mapa de registros opera sin anomalías, cumpliendo el OE2 con un canal confiable entre CPU y hardware dedicado.

4.2.3 Prueba de integración del SoC (OE3)

La prueba integral del SoC se comprobó mediante la ejecución exitosa de la aplicación de evaluación exhaustiva (`test_app2`). Esta prueba demostró la correcta interoperabilidad de todos los componentes del sistema trabajando en conjunto: el procesador MicroBlaze V ejecutando el *firmware* desde la memoria BRAM, el bloque AXI SmartConnect enrutando exitosamente las transacciones en el bus, el módulo PLPUF respondiendo a las peticiones en hardware y el controlador UART transmitiendo los resultados hacia el computador anfitrión.

La ejecución autónoma del protocolo permitió recolectar aproximadamente 140 000 respuestas del sistema de forma ininterrumpida. La estabilidad operacional lograda a lo largo de las 10 corridas independientes evidencia una comunicación bidireccional USB-UART con cero pérdida de información. Esta prueba de resistencia (*end-to-end*) constituye la evidencia definitiva del logro exitoso del OE3 y confirma la conformación de un sistema embebido estable, apto para el prototipado en seguridad de la información.

4.2.4 Resultados de la evaluación de seguridad (OE4)

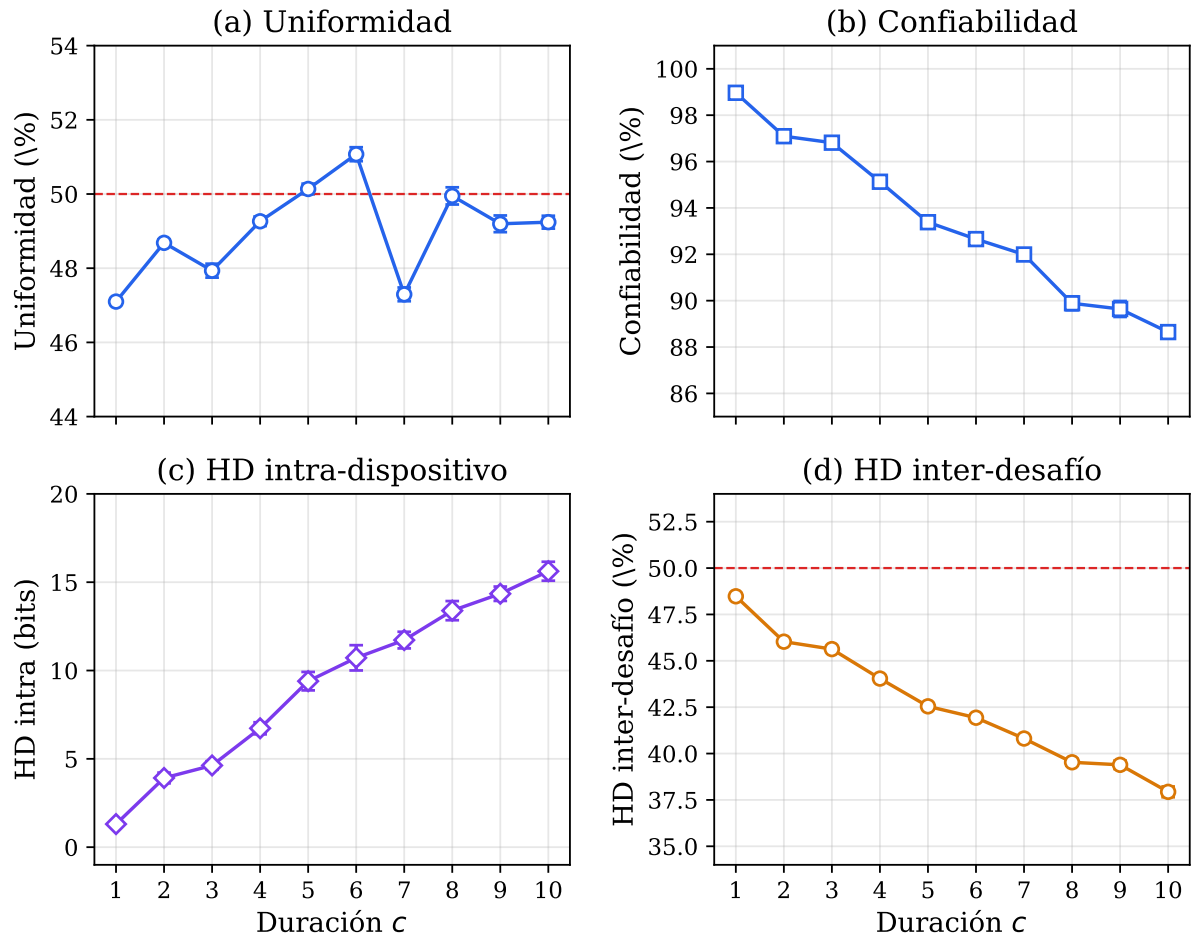
Es importante señalar que todas las pruebas corresponden a una evaluación **intra-dispositivo**: se realizaron sobre un único chip FPGA (Artix-7 XC7A35T-1CPG236C). La limitación de no contar con placas adicionales fue declarada en la Sección 1. La unicidad se estima indirectamente mediante la distancia de Hamming inter-desafío. Los valores reportados a continuación corresponden a la media $\pm$ desviación estándar calculadas sobre las 10 corridas independientes.

Resultados del barrido de duración

La Tabla 4.3 resume los resultados del barrido de duración para las cinco métricas principales. La Figura 4.1 presenta estos mismos datos de forma gráfica.

Tabla 4.3: Resultados del barrido de duración (media $\pm$ desviación estándar, $n = 10$ corridas, 20 desafíos $\times$ 50 repeticiones por corrida).

c	Unif. (%)	Conf. (%)	BER (%)	Intra-HD (bits)	Inter-HD (%)
1	47.10 ± 0.04	98.97 ± 0.03	1.03 ± 0.03	1.31 ± 0.06	48.48 ± 0.06
2	48.68 ± 0.09	97.09 ± 0.07	2.91 ± 0.07	3.92 ± 0.30	46.03 ± 0.07
3	47.93 ± 0.19	96.81 ± 0.06	3.19 ± 0.06	4.63 ± 0.24	45.63 ± 0.10
4	49.27 ± 0.13	95.13 ± 0.15	4.87 ± 0.15	6.73 ± 0.34	44.04 ± 0.16
5	50.14 ± 0.14	93.38 ± 0.13	6.62 ± 0.13	9.40 ± 0.52	42.55 ± 0.14
6	51.07 ± 0.19	92.66 ± 0.23	7.34 ± 0.23	10.72 ± 0.72	41.93 ± 0.16
7	47.30 ± 0.19	91.99 ± 0.12	8.01 ± 0.12	11.72 ± 0.47	40.81 ± 0.17
8	49.95 ± 0.23	89.89 ± 0.30	10.11 ± 0.30	13.39 ± 0.54	39.53 ± 0.18
9	49.20 ± 0.22	89.64 ± 0.34	10.36 ± 0.34	14.35 ± 0.41	39.39 ± 0.23
10	49.24 ± 0.17	88.65 ± 0.23	11.36 ± 0.23	15.62 ± 0.54	37.93 ± 0.29

**Figura 4.1:** Resumen de métricas del PLPUF en función de la duración de activación c . Las barras de error representan $\pm 1\sigma$ sobre 10 corridas independientes. (a) Uniformidad, (b) confiabilidad, (c) HD intra-dispositivo, (d) HD inter-desafío.

Uniformidad

La uniformidad mide la proporción de unos en la respuesta *golden* del PLPUF; el valor ideal es 50 %, lo que indica máxima entropía por bit [22]. Como se muestra en la Figura 4.2, la uniformidad promedio se mantiene entre 47.10 % ($c = 1$) y 51.07 % ($c = 6$) a lo largo de todas las duraciones de activación, con una desviación estándar inferior a 0.25 puntos porcentuales. Este comportamiento confirma la ausencia de sesgos sistemáticos en el hardware, comprobando el requerimiento analizado en la Sección 4.2.1.

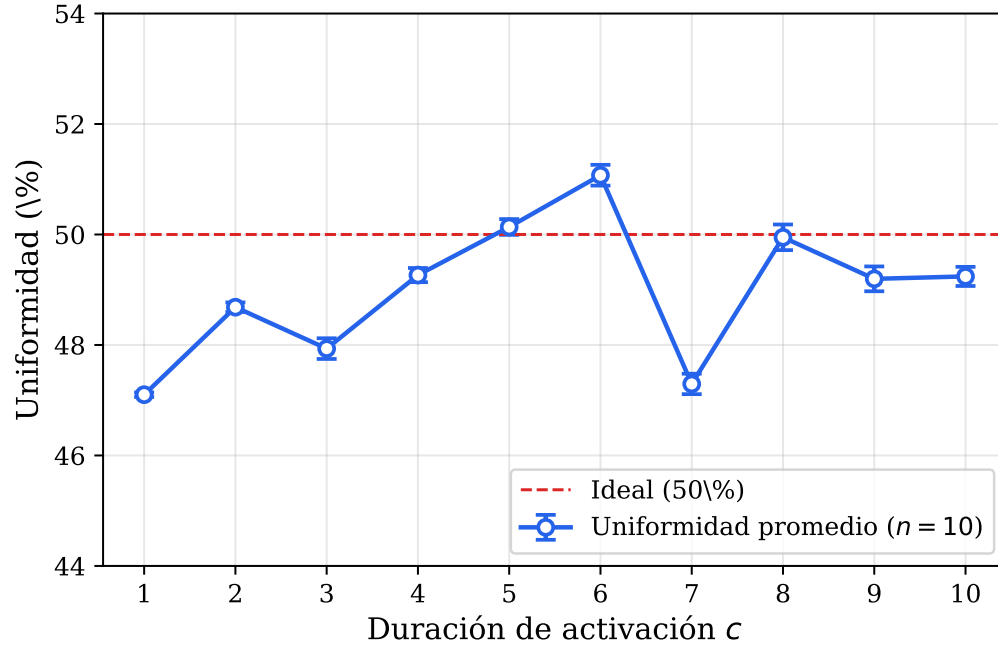


Figura 4.2: Uniformidad del PLPUF en función de la duración de activación c (media $\pm 1\sigma$, $n = 10$). La línea discontinua roja indica el valor ideal de 50 %.

Confiabilidad y tasa de error de bit (BER)

La confiabilidad cuantifica la reproducibilidad de las respuestas ante el mismo desafío [22]. La Figura 4.3 muestra la evolución de ambas métricas en función de la duración de activación. Se observa una relación inversa: para $c = 1$, la confiabilidad alcanza 98.97 ± 0.03 %, mientras que para $c = 10$ desciende a 88.65 ± 0.23 %. Para $c \leq 4$, la confiabilidad se mantiene por encima del 95 %, umbral considerado aceptable para identificación de dispositivos [4]. La configuración $c = 1$ resulta ser la más confiable con un BER de 1.03 ± 0.03 %.

La degradación con el aumento de c se explica por la naturaleza del lazo combinacional: la oscilación prolongada causa una acumulación exponencial de ruido térmico en el circuito lógico, perturbando la evaluación final y generando una menor fiabilidad intrínseca [4].

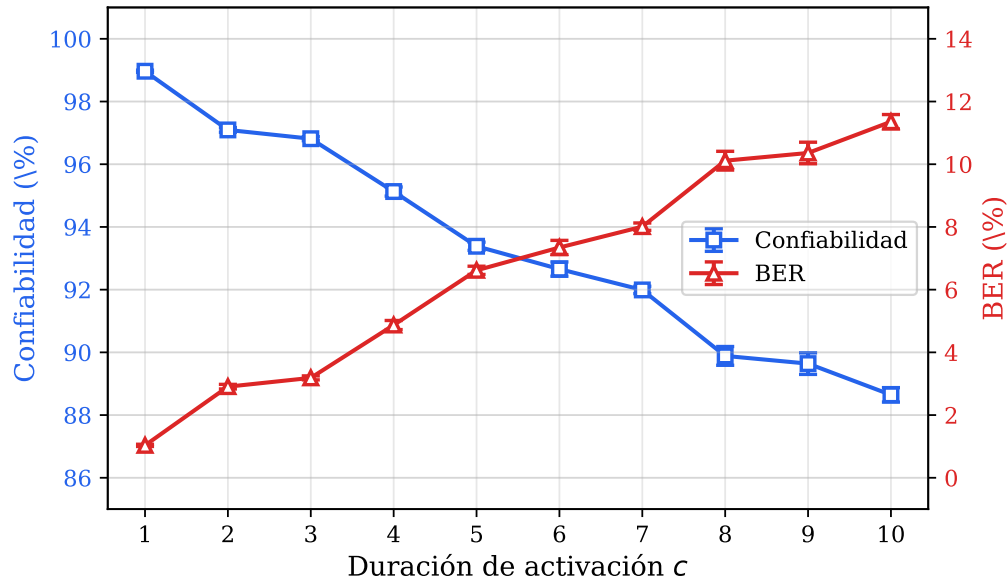


Figura 4.3: Confiabilidad (eje izquierdo) y BER (eje derecho) en función de la duración de activación c (media $\pm 1\sigma$, $n = 10$).

Distancia de Hamming intra-dispositivo e inter-desafío

La distancia de Hamming (HD) intra-dispositivo ideal es 0 bits, representando reproducibilidad perfecta. Para $c = 1$, la HD intra-dispositivo es de 1.31 ± 0.06 bits (1.02 % de la respuesta), consistente con la literatura. A medida que c aumenta, crece hasta 15.62 ± 0.54 bits ($\sim 12.2\%$) para $c = 10$ (Figura 4.4).

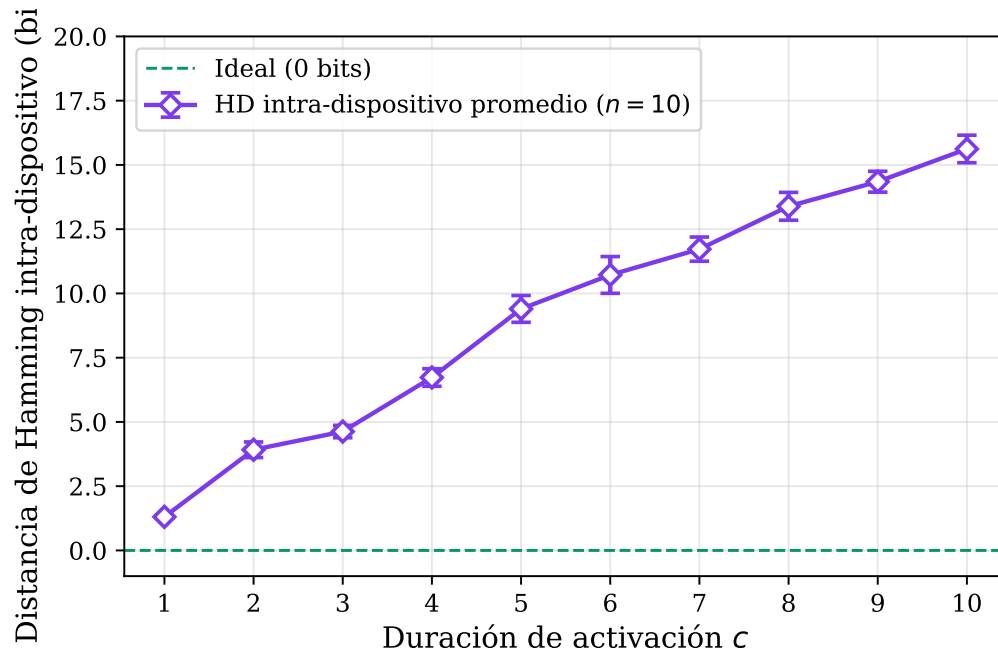


Figura 4.4: Distancia de Hamming intra-dispositivo promedio en función de la duración de activación c (media $\pm 1\sigma$, $n = 10$).

La distancia de Hamming inter-desafío (ideal 50 %) se empleó como alternativa evaluativa a la carencia de métricas multi-chip. Como muestra la Figura 4.5, en $c = 1$ fue de $48.48 \pm 0.06 \%$. Con el aumento de c disminuye a $37.93 \pm 0.29 \%$ en $c = 10$. Esto implica que a mayores duraciones, los patrones lógicos tienden a colisionar y reducir la diversidad criptográfica en la firma de salida del PLPUF.

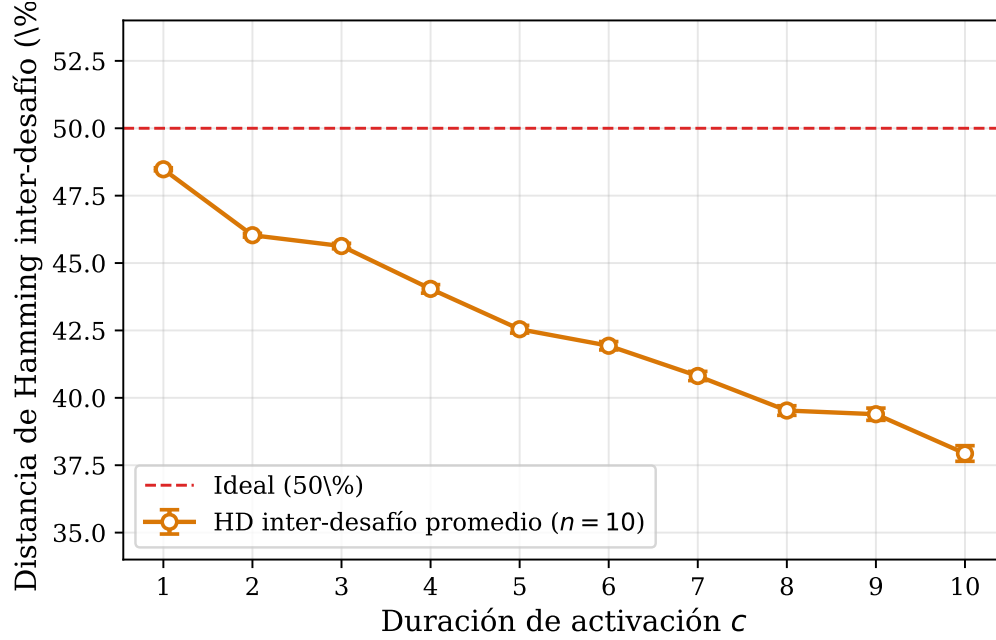


Figura 4.5: Distancia de Hamming inter-desafío (como porcentaje de los 128 bits) en función de la duración de activación c (media $\pm 1\sigma$, $n = 10$). La línea discontinua roja indica el ideal de 50 %.

Análisis detallado a $c = 5$ y efecto avalancha

La duración $c = 5$ se seleccionó como punto representativo de análisis medio. Un análisis del mapa de estabilidad sobre las 10 corridas indicó un promedio de 39.5 ± 3.8 bits altamente estables y apenas 0.5 ± 0.5 bits sistemáticamente ruidosos, requiriendo algoritmos correctores (Helper Data) para estabilización determinista en aplicaciones prácticas. El histograma de distancia no excedió los 29 bits de error bajo ninguna evaluación de prueba.

Respecto al efecto avalancha (Figura 4.6), el porcentaje promedio observado de cambio en los bits de respuesta al alternar un solo bit del desafío de entrada fue del $8.43 \pm 1.15 \%$, inferior al 50 % requerido por la criptografía moderna. Este bajo grado de difusión es atribuible a las limitantes arquitectónicas del polinomio matemático de estructura Fibonacci implementado [9].

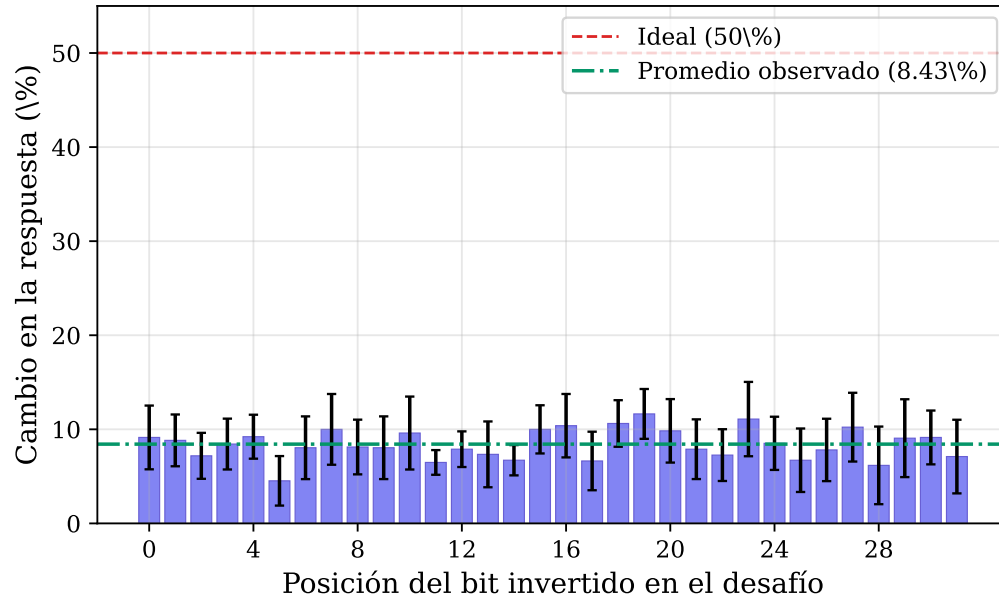


Figura 4.6: Efecto avalancha del PLPUF a $c = 5$: porcentaje de bits que cambian en la respuesta al invertir un solo bit del desafío (media $\pm 1\sigma$, $n = 10$). La línea discontinua roja indica el ideal de 50 % y la línea punteada verde el promedio observado.

4.2.5 Comparación con referencias y discusión general

La Tabla 4.4 enmarca los resultados de la plataforma actual frente a los hallazgos iniciales documentados por Hori et al. [4].

Tabla 4.4: Comparación de resultados con la evaluación de referencia de Hori et al. [4].

Parámetro	Este trabajo	Hori et al. [4]
Plataforma FPGA	Artix-7 (XC7A35T)	Virtex-5 (XC5VLX30)
Frecuencia de reloj	100 MHz	24 MHz
Ancho del PLPUF	128 bits	128 bits
Dispositivos evaluados	1	16
Desafíos por dispositivo	20	100
Repeticiones por CRP	50	100
HD intra ($c = 1$)	1.31 bits	1.76 bits
Confiabilidad ($c = 1$)	98.97 %	~98.6 %
HD inter-desafío ($c = 1$)	48.48 %	—
HD inter-dispositivo	No evaluada	~50 %

Los hallazgos demuestran una alta concordancia con las capacidades teóricas del diseño. La distancia de error promedio (1.31 bits) es inclusive menor que los 1.76 bits en [4], infiriendo que la familia Artix-7 a 100 MHz produce una mayor resiliencia ante el en-

torno. La repetibilidad es alta y confirma empíricamente la viabilidad de utilización para arquitecturas ligeras de Hardware de Confianza (Hardware Root of Trust).

Los resultados de este capítulo constatan de manera ineludible que la duración de la activación (el parámetro ' c ') actúa como el modulador de control entre seguridad de entropía y consistencia operacional. Las operaciones en torno a $c = 1$ resultan óptimas para tareas criptográficas, mientras que valores ascendentes erosionan el espacio de distinción de la firma resultante.

La principal limitación identificada fue la falta de evaluación inter-dispositivo en un grupo poblacional, lo cual impidió una cuantificación rigurosa del alias de bit como señalan las directrices de Maiti y Wilde [22, 23]. Sin embargo, este ejercicio demostró que a nivel constructivo, desde su descripción y control por un SoC empotrado, un módulo PLPUF abierto resulta factible y ofrece las propiedades mínimas de aleatoriedad local necesarias.

Capítulo 5

Conclusiones y recomendaciones

Este capítulo presenta las conclusiones derivadas del trabajo realizado, en términos del cumplimiento de los objetivos planteados, los hallazgos experimentales y el aporte global del proyecto a la línea de investigación en seguridad de hardware sobre FPGA. A continuación se exponen las recomendaciones de trabajo futuro orientadas a superar las limitaciones identificadas y a extender las capacidades del módulo desarrollado.

5.1 Conclusiones

El presente proyecto logró diseñar, implementar e integrar un módulo PLPUF programable dentro de un SoC basado en FPGA, cumpliendo satisfactoriamente el objetivo general propuesto. El módulo opera como periférico AXI4-Lite controlado por el procesador suave MicroBlaze V sobre la plataforma Basys 3 con FPGA Artix-7, proporcionando una solución funcional, abierta y reproducible para la generación de identificadores de hardware basados en variaciones físicas del silicio. La integración alcanzada cubre todas las capas del sistema, desde la descripción HDL del núcleo PUF hasta el *firmware* de evaluación y el *dataset* resultante, lo cual constituye el principal aporte diferenciador respecto a las implementaciones aisladas reportadas en la literatura [13, 14].

El núcleo PLPUF fue implementado exitosamente en Verilog con ancho parametrizable, configurado a 128 bits, empleando una cadena de inversores con retroalimentación tipo Fibonacci y una máquina de estados finitos de cuatro estados. La síntesis en AMD Vivado generó el *bitstream* para la FPGA Artix-7, y las pruebas de validación funcional confirmaron que el módulo genera respuestas de 128 bits que varían con el desafío y la duración de activación, cumpliendo así el primer objetivo específico. El uso de los atributos de síntesis `DONT_TOUCH` y `ALLOW_COMBINATORIAL_LOOPS` resultó esencial para preservar la integridad del lazo combinacional durante la optimización del sintetizador.

La interfaz AXI4-Lite *slave*, con un mapa de 11 registros de 32 bits para control, duración, desafío, respuesta y estado, operó correctamente en las 10 corridas de evaluación ejecutadas. La verificación de lectura y escritura de todos los registros, incluida la condición de

solo lectura de los registros de respuesta, validó la integridad del protocolo de comunicación entre el procesador y el periférico, satisfaciendo el segundo objetivo específico.

La integración completa del SoC en Vivado IP Integrator, incluyendo MicroBlaze V, memoria BRAM de 64 KB, controlador UART, módulo de depuración MDM, generador de reloj y el periférico PLPUF, demostró ser estable y funcional. El desarrollo del *firmware* de control en C mediante AMD Vitis, junto con el *driver* de bajo nivel y las dos aplicaciones de prueba, permitió la ejecución autónoma de más de 140 000 evaluaciones del PLPUF distribuidas en 10 corridas independientes, con comunicación estable a través de la UART. Este resultado confirma el cumplimiento del tercer objetivo específico.

La evaluación del PLPUF mediante un protocolo exhaustivo basado en las métricas estándar de la literatura [4, 22] demostró que el módulo exhibe propiedades estadísticas coherentes con las esperadas para esta arquitectura. La uniformidad se mantuvo entre 47 % y 51 % para todas las duraciones de activación, indicando ausencia de sesgo estructural significativo [22]. La confiabilidad superó el 95 % para duraciones $c \leq 4$, con un máximo de 98.97 % para $c = 1$, valor comparable al reportado por Hori et al. [4] para FPGAs Virtex-5. La distancia de Hamming inter-desafío alcanzó el 48.48 % para $c = 1$, cercana al ideal de 50 %, lo que indica que el PLPUF genera respuestas suficientemente diversas ante desafíos distintos. Estos resultados validan la correcta portabilidad de la arquitectura a la familia Artix-7 y satisfacen el cuarto objetivo específico.

Los datos experimentales confirmaron que la duración de activación c constituye el parámetro más crítico del PLPUF, al gobernar el compromiso fundamental entre confiabilidad y entropía. La configuración $c = 1$ ofrece la mejor combinación de métricas, mientras que valores de $c \geq 5$ reducen la confiabilidad por debajo del umbral comúnmente aceptado [22]. Esta observación es consistente con los riesgos documentados por Alsharkawy et al. [9] respecto a duraciones elevadas.

El efecto avalancha promedio de 8.43 % se situó significativamente por debajo del ideal de 50 %, lo cual constituye una limitación inherente a la topología de retroalimentación Fibonacci del PLPUF [9]. Esta restricción debe considerarse en el diseño de protocolos criptográficos que dependan de alta difusión entre el desafío y la respuesta.

La alta reproducibilidad entre las 10 corridas independientes, con desviaciones estándar inferiores a un punto porcentual en todas las métricas, valida tanto la estabilidad del módulo hardware como la robustez del protocolo de evaluación implementado. No obstante, la evaluación se limitó a un único dispositivo FPGA, por lo que no fue posible calcular la métrica de unicidad inter-dispositivo [23]. La distancia de Hamming inter-desafío se utilizó como indicador indirecto de la capacidad de diferenciación del PLPUF.

En síntesis, el proyecto cumplió con todos los objetivos propuestos y generó una plataforma completa, desde el diseño HDL hasta el software de evaluación y el *dataset* de pares desafío-respuesta, que puede servir como base para futuras investigaciones en seguridad de hardware sobre FPGAs en entornos académicos y de investigación. El aporte del trabajo se sitúa en tres ejes principales: en el eje técnico, al ofrecer un núcleo IP parametrizable y

reutilizable para la familia Artix-7 que no se identificó previamente en la literatura accesible; en el eje metodológico, al sistematizar un protocolo de evaluación intra-dispositivo reproducible basado en métricas estándar; y en el eje educativo, al proveer una plataforma abierta sobre la cual estudiantes e investigadores pueden experimentar con primitivas de seguridad de hardware sin depender de soluciones comerciales costosas o cerradas.

5.2 Recomendaciones

A partir de las conclusiones expuestas, y considerando tanto las limitaciones declaradas como las oportunidades identificadas a lo largo del desarrollo, se proponen a continuación las siguientes recomendaciones para trabajos futuros. Estas se presentan ordenadas según su prioridad estimada en términos del valor añadido al estado actual del módulo.

Se recomienda, en primer lugar, replicar las pruebas sobre múltiples placas Basys 3, u otras placas con FPGA Artix-7, para calcular la unicidad inter-dispositivo y el alias de bit con intervalos de confianza estadísticamente robustos, tal como sugieren Wilde y Pehl [23]. Esta evaluación permitiría cuantificar la capacidad real del PLPUF para distinguir chips individuales en escenarios de autenticación y constituiría el complemento natural más urgente del trabajo realizado.

La evaluación realizada se ejecutó a temperatura ambiente y voltaje nominal. Para validar la confiabilidad del PLPUF en aplicaciones prácticas, es necesario caracterizar su comportamiento bajo variaciones de temperatura y fluctuaciones de voltaje, siguiendo las recomendaciones de Maiti et al. [22].

Dado que la confiabilidad se degrada progresivamente para duraciones de activación $c \geq 2$, la incorporación de algoritmos de datos auxiliares (*Helper Data Algorithms*) con códigos correctores de errores [20] permitiría derivar claves criptográficas determinísticas a partir de las respuestas del PLPUF, incluso en configuraciones con BER moderado.

El bajo efecto avalancha observado está directamente relacionado con la estructura del polinomio primitivo utilizado, que posee solo tres puntos de retroalimentación XOR. Se recomienda evaluar polinomios con mayor número de taps o funciones de retroalimentación no lineales, como propusieron Huang et al. [11], para mejorar la difusión entre desafío y respuesta. Dado que el diseño es parametrizable y está empaquetado como núcleo IP, su portabilidad a familias FPGA de mayor capacidad, como Kintex-7 o UltraScale+, permitiría evaluar la influencia de la tecnología de fabricación sobre las métricas del PLPUF. Asimismo, resulta de interés explorar la compatibilidad con procesadores suaves alternativos al MicroBlaze V.

Finalmente, el módulo PLPUF y la infraestructura SoC desarrollados proveen la base necesaria para implementar protocolos de autenticación ligeros basados en PUFs, como los propuestos por Idriss et al. [16] y Hou et al. [17]. Se recomienda utilizar la plataforma como banco de pruebas para evaluar la viabilidad práctica de estos protocolos en sistemas embebidos con recursos limitados.

Anexo A

Implicaciones sociales y éticas del proyecto

La seguridad no es un concepto abstracto. Detrás de cada dispositivo conectado a una red hay una persona, una familia, una institución o una comunidad que confía en que sus datos, su identidad y su integridad estarán protegidos. Cuando un ingeniero diseña un módulo de seguridad para sistemas embebidos, no está simplemente manipulando compuertas lógicas y registros: está asumiendo una responsabilidad directa sobre la confianza que otros depositarán en ese hardware. Este proyecto me confrontó con esa realidad desde el primer día.

El Internet de las Cosas ha transformado la vida cotidiana de formas que hace dos décadas habrían parecido ciencia ficción. Dispositivos médicos que monitorean signos vitales, sensores industriales que controlan procesos críticos, cerraduras inteligentes que protegen hogares y sistemas vehiculares que toman decisiones en fracciones de segundo. Todos ellos comparten una característica común: operan en entornos físicamente accesibles, donde un atacante con las herramientas adecuadas puede intentar extraer claves, clonar identidades o manipular el comportamiento del dispositivo. La consecuencia de una falla de seguridad en estos contextos no se mide solamente en pérdidas económicas, sino en riesgos reales para la salud, la privacidad y la seguridad de las personas.

Las Funciones Físicamente No Clonables representan un cambio de paradigma en la forma de abordar la seguridad del hardware. Al eliminar la necesidad de almacenar claves criptográficas en memorias que pueden ser leídas o copiadas, las PUFs ofrecen una capa de protección fundamentada en la física misma del silicio, imposible de replicar incluso por el fabricante original. Sin embargo, esta promesa tecnológica conlleva una responsabilidad ética significativa. Quien diseña e implementa una PUF debe ser riguroso en la evaluación de sus propiedades estadísticas, honesto en la comunicación de sus limitaciones y consciente de que una evaluación incompleta o sesgada puede generar una falsa sensación de seguridad con consecuencias potencialmente graves.

Durante el desarrollo de este proyecto, una de las decisiones más difíciles fue aceptar

y documentar abiertamente las limitaciones de la evaluación realizada. La ausencia de pruebas inter-dispositivo impide afirmar con certeza que el módulo PLPUF implementado sea capaz de distinguir chips individuales en un escenario de autenticación real. Habría sido tentador omitir esta limitación o minimizar su importancia, pero hacerlo habría comprometido la integridad del trabajo y, potencialmente, la seguridad de quienes en el futuro pudieran utilizar este módulo. La honestidad intelectual no es un lujo en ingeniería de seguridad: es un requisito ético fundamental.

Otro aspecto que merece reflexión es la decisión de desarrollar este proyecto como una solución de código abierto. En el ámbito de la seguridad informática existe un debate permanente entre quienes abogan por la seguridad mediante la oscuridad y quienes defienden la transparencia como principio fundamental. Mi convicción, respaldada por décadas de experiencia en la comunidad criptográfica, es que un diseño de seguridad solo puede considerarse confiable cuando ha sido sometido al escrutinio público. Al publicar el código HDL, los controladores de software y los datos de evaluación, este proyecto invita a la comunidad académica a verificar, reproducir y mejorar los resultados. Esta apertura no debilita la seguridad; la fortalece.

No obstante, la publicación de herramientas y conocimientos relacionados con la seguridad del hardware plantea inevitablemente el dilema del uso dual. El mismo conocimiento que permite diseñar una PUF robusta puede emplearse para identificar sus debilidades y explotarlas. Este dilema no es exclusivo de este proyecto ni de este campo; es inherente a toda la investigación en seguridad. La postura que adopto es que la contribución neta del conocimiento abierto es positiva: los defensores siempre necesitan más información que los atacantes, y la transparencia permite que la comunidad identifique y corrija vulnerabilidades antes de que sean explotadas.

Desde una perspectiva social más amplia, este proyecto contribuye a la democratización de la seguridad del hardware. Las soluciones comerciales de seguridad en silicio suelen estar protegidas por patentes y licencias que las hacen inaccesibles para instituciones educativas, pequeñas empresas y países en desarrollo. Al proporcionar una plataforma completa, documentada y gratuita, se abre la posibilidad de que estudiantes e investigadores de cualquier parte del mundo puedan experimentar con técnicas de seguridad de hardware sin depender de costosas licencias o hardware propietario. En un contexto como el costarricense, donde la formación de ingenieros capaces de diseñar sistemas seguros es una necesidad estratégica, esta contribución adquiere un valor que trasciende lo puramente técnico.

Finalmente, deseo reflexionar sobre el rol del ingeniero en la sociedad digital contemporánea. Vivimos en una era en la que la confianza en la tecnología es un pilar de la convivencia social. Cada vez que una persona utiliza un dispositivo conectado, está depositando implícitamente su confianza en los ingenieros que lo diseñaron, fabricaron y programaron. Esta confianza no debe tomarse a la ligera. Mi deber como ingeniero en computadores no se agota en la correcta implementación de un circuito o la optimización de una métrica: se extiende a la responsabilidad de garantizar que mi trabajo contribuya

al bienestar de las personas y no se convierta, por negligencia o deshonestidad, en un instrumento de vulnerabilidad. Con este proyecto, he intentado honrar esa responsabilidad con rigor técnico, transparencia en los resultados y compromiso con la comunidad que algún día podría beneficiarse de este trabajo.

Bibliografía

- [1] AMD, *MicroBlaze V Processor Reference Guide*, <https://docs.amd.com/r/en-US/ug1629-microblaze-v-reference>, 2025, v2025.1.
- [2] Arm Ltd., *AMBA AXI and ACE Protocol Specification*, <https://developer.arm.com/documentation/ih0022/hc>, 2021.
- [3] Digilent Inc., *Basys 3 FPGA Board Reference Manual*, <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>, 2014, rev. C.
- [4] Y. Hori, H. Kang, T. Katashita, and A. Satoh, “Pseudo-lfsr puf: A compact, efficient and reliable physical unclonable function,” in *2011 International Conference on Reconfigurable Computing and FPGAs (ReConFig)*. IEEE, 2011. [Online]. Available: <https://ieeexplore.ieee.org/document/6128581>
- [5] A. Shamsoshoara, A. Korenda, F. Afghah, and S. Zeadally, “A survey on physical unclonable function (puf)-based security solutions for internet of things,” *arXiv preprint arXiv:1907.12525*, 2020. [Online]. Available: <https://arxiv.org/pdf/1907.12525>
- [6] S. Hou, Y. Guo, S. Li, D. Deng, and Y. Lei, “A lightweight and secure-enhanced strong puf design on fpga,” *Electronics Express*, 2019. [Online]. Available: https://www.researchgate.net/publication/337691693_A_lightweight_and_secure-enhanced_Strong_PUF_design_on_FPGA
- [7] H. Handschuh, G.-J. Schrijen, and P. Tuyls, “Hardware intrinsic security from physically unclonable functions,” in *Towards Hardware-Intrinsic Security*, ser. Information Security and Cryptography, A.-R. Sadeghi and D. Naccache, Eds. Springer-Verlag Berlin Heidelberg, 2011. [Online]. Available: https://www.researchgate.net/publication/226577915_Hardware_Intrinsic_Security_from_Physically_Unclonable_Functions
- [8] Instituto Tecnológico de Costa Rica, “Tecnológico de costa rica: Información institucional,” <https://universidades.cr/universidades/tecnologico-de-costa-rica-tec>, 2024.
- [9] M. Alsharkawy, J. Zwerschke, H. Nassar, J. González-Gómez, and J. Henkel, “Late breaking results: The hidden risks of activation duration in plpufs,”

2024. [Online]. Available: https://www.researchgate.net/publication/395614756_Late_Breaking_Results_The_Hidden_Risks_of_Activation_Duration_in_PLPUFs
- [10] M. K. J. K. M. Srilakshmi, BVDN. S., “Designing a strong physically unclonable function using low power lfsr,” *Recent Developments in Electronics and Communication Systems*, 2023. [Online]. Available: <https://ebooks.iospress.nl/doi/10.3233/ATDE221237>
- [11] Z. Huang, X. Du, C. Su, and J. Bian, “Nlfsr puf: A lightweight and reliable strong puf with modeling-attack resistance,” *Microelectronics Journal*, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1879239126000937>
- [12] W. Liu, L. Zhang, Z. Zhang, C. Gu, C. Wang, M. O’Neill, and F. Lombardi, “Xor-based low-cost reconfigurable pufs for iot security,” *ACM Transactions on Embedded Computing Systems (TECS)*, 2019. [Online]. Available: <https://dl.acm.org/doi/10.1145/3274666>
- [13] K. Lata and L. R. Cenkeramaddi, “Fpga-based puf designs: A comprehensive review and comparative analysis,” *Cryptography*, 2023. [Online]. Available: <https://www.mdpi.com/2410-387X/7/4/55>
- [14] N. N. Anandakumar, M. S. Hashmi, and M. Tehranipoor, “Fpga-based physical unclonable functions: A comprehensive overview of theory and architectures,” *INTEGRATION, the VLSI journal*, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167926021000766>
- [15] S. Chen, B. Li, and C. Zhou, “Fpga implementation of sram pufs based cryptographically secure pseudo-random number generator,” *Microprocessors and Microsystems*, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0141933116303441>
- [16] T. A. Idriss, H. A. Idriss, and M. A. Bayoumi, “A lightweight puf-based authentication protocol using secret pattern recognition for constrained iot devices,” *IEEE Access*, 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9444356>
- [17] S. Hou, Y. Ma, D. Deng, Z. Wang, and G. Ren, “Modeling and physical attack resistant authentication protocol with double pufs,” *Journal of Information Security and Applications*, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214212623001278>
- [18] R. A. Alhamarneh and M. M. Singh, “Strengthening internet of things security: Surveying physical unclonable functions for authentication, communication protocols, challenges, and applications,” *Applied Sciences*, 2024. [Online]. Available: <https://www.mdpi.com/2076-3417/14/5/1700>

- [19] P. Sudhanya and P. M. Krishnammal, “Study of different silicon physical unclonable functions,” in *2016 IEEE Wireless Information Technology and Systems (WiSPNET)*. IEEE, 2016. [Online]. Available: https://www.researchgate.net/publication/308708405_Study_of_different_silicon_Physical_Unclonable_Functions
- [20] J. Delvaux, D. Gu, D. Schellekens, and I. Verbauwhede, “Helper data algorithms for puf-based key generation: Overview and analysis,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2015. [Online]. Available: <https://ieeexplore.ieee.org/document/6965637>
- [21] F. Farha, H. Ning, H. Liu, L. T. Yang, and L. Chen, “Physical unclonable functions based secret keys scheme for securing big data infrastructure communication,” *Information Sciences*, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S002002551930605X>
- [22] A. Maiti, V. Gunreddy, and P. Schaumont, “A systematic method to evaluate and compare the performance of physical unclonable functions,” in *Constructive Side-Channel Analysis and Unified Engineering of Obfuscation*. Springer, New York, NY, 2012. [Online]. Available: https://link.springer.com/chapter/10.1007/978-1-4614-1362-2_11
- [23] F. Wilde and M. Pehl, “On the confidence in bit-alias measurement of physical unclonable functions,” *arXiv preprint arXiv:2001.07660*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.07660>
- [24] AMD, *Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator*, <https://docs.amd.com/r/en-US/ug994-vivado-ip-subsystems>, 2025, v2025.1.
- [25] —, *MicroBlaze V Processor Embedded Design User Guide*, <https://docs.amd.com/r/en-US/ug1711-microblaze-v-embedded-design>, 2025, v2025.1.
- [26] Xilinx, *Vivado Design Suite — AXI Reference Guide*, <https://docs.amd.com/r/en-US/ug1037-vivado-axi-reference-guide>, 2017, v4.0.
- [27] AMD, *Embedded Design Development Using Vitis User Guide*, <https://docs.amd.com/r/en-US/ug1701-vitis-embedded>, 2025, v2025.1.
- [28] T. Idriss, H. Idriss, and M. Bayoumi, “A puf-based paradigm for iot security,” in *2016 IEEE 3rd World Forum on Internet of Things (WF-IoT)*. IEEE, 2016. [Online]. Available: https://www.researchgate.net/publication/313543781_A_PUF-based_paradigm_for_IoT_security
- [29] R. Parikh and K. Parikh, “Survey on hardware security: Pufs, trojans, and side-channel attacks,” *Preprints.org*, 2025. [Online]. Available: <https://www.preprints.org/manuscript/202501.1559.v1>

-
- [30] A. Al-Meer and S. Al-Kuwari, “Physical unclonable functions (puf) for iot devices,” *ACM Computing Surveys*, 2023. [Online]. Available: <https://doi.org/10.1145/3591464>
- [31] C. Frisch, F. Wilde, T. Holzner, and M. Pehl, “A practical approach to estimate the min-entropy in pufs,” *Journal of Hardware and Systems Security*, 2023. [Online]. Available: <https://link.springer.com/article/10.1007/s41635-023-00139-x>